

预约表感知派遣的闭环双层框架: 柔性作业车间调度与 AGV 无冲突路由一体化优化

柔性作业车间的加工与运输一体化调度几乎总是把行驶时间建模为按位置索引的常数矩阵, 这等于默认车辆之间从不相互干扰。一旦有限规模的车队共享稀疏的走廊网络, 行驶时间便不再是输入数据, 而是路由决策的结果: 延误成为一个工件施加给另一个工件的量, 而非布局的固有属性。本文研究这样一类一体化问题: 每道工序可由若干台能力重叠、效率互异的机械臂之一加工, 同时 AGV 必须在该网络上无冲突行驶。使这一耦合真正产生后果的是异构性——当通往快速机械臂的走廊拥堵时, 一台较慢但连通性更好的机械臂可能是更优指派。该权衡在两类最接近的工作中均无法表述: 一类具备机器柔性, 但上层在常数运输时间矩阵上排产, 看不见拥堵; 另一类具备真正的无冲突路由, 却沿用每道工序仅一台可选机器的经典基准, 指派根本不成为决策。

本文提出闭环双层框架: 上层决定机器指派与工序排序, 下层以时间窗预约生成无冲突路径, 两层沿评价与决策两条通路连接。评价通路把每一代每个候选解的适应度定义为下层实际路由之后的完工时间, 使让行延误进入搜索所优化的目标, 而不再是事后校验; 决策通路的核心机制是预约表感知的车辆派遣——通过对预约表做两段式试探来选车, 取代以理想最短路估计到达时刻的规则派车。为使该机制在同等算力下付得起, 本文给出两项可证等价的降本: 可采纳下界剪枝与胜者路径复用。

在 10 个争用强度标定于 6.6% 至 33.5% 的受控算例上, 以 10 个随机种子、同挂钟预算与配对 Wilcoxon 检验, 与一个四级基线阶梯比较, 其中前两级分别对应两篇已发表工作的结构。把拥堵搬进搜索目标使完工时间降低 18.86% ($p = 0.0000$); 在闭环内部将规则派车替换为预约表感知派遣再降低 2.45% ($p = 0.0054$); 端到端相对文献结构降低 21.35%(94 胜 5 负, $p = 0.0000$)。

两条机制都显著, 但角色不同: 闭环是主效应 (18.86%/17.75%), 预约表感知派遣是有适用上界的次效应——开环下它值 4.59% ($p = 0.0000$), 闭环内平均再降 2.45% ($p = 0.0054$), 而在车队偏大的两格上它会被自身开销反噬。端到端 21.35% 接近两者独立时应有的 22.6%, 呈近加性而非正交互; 开环试探单独远不能代替闭环。本文并以三个受控负对照 (走廊定价、凭证制导打分、穷举式真解码) 说明为何在若干更具雄心的接口设计中选定了这一个。

Additional Key Words and Phrases: 柔性作业车间调度, AGV 无冲突路由, 一体化调度, 闭环双层框架, 预约表感知派遣, 异构机械臂, 走廊争用, 完工时间

ACM Reference Format:

. 2025. 预约表感知派遣的闭环双层框架: 柔性作业车间调度与 AGV 无冲突路由一体化优化. 1, 1 (August 2025), 32 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 引言

1.1 研究背景与动机

考虑运输的柔性作业车间调度长期建立在一个便利的假设之上: 工件在两台机器之间移动所需的时间是一个常数, 可从按位置对索引的矩阵中读出。这是文献中的主流约定 [9], 近期的模型仍普遍沿用 [1, 5, 6]。当车队规模相对于通道网络足够小、车辆彼此很少相遇时, 该假设是合理的。然而实际车间并非如此构造: 少量车辆共享一组稀疏通道, 单车道路段无法在同一时刻被双向通行; 一旦车队密度足够高, 车队自身的交通便从次要干扰上升

Author's Contact Information:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

为一阶的延误来源 [3]。承认这一点之后, 行驶时间就不再是数据, 而成为两个决策的结果——车辆走哪条路, 以及谁先预约了被争用的路段。由此产生的延误是一个工件施加给另一个工件的, 而不是布局的固有属性。

这一点只有在排程对此有所作为时才对调度有意义, 而机器柔性恰好提供了作为的余地。当一道工序可由某个可选集合中的任一机械臂加工、且加工时间取决于所选机械臂时, 指派决策同时选定了目的地与工时。设想一道工序既可交给车间远端一台快速机械臂, 也可交给紧邻上下料站的一台较慢机械臂。在常数运输时间矩阵下, 这个比较是算术性的: 较短加工加较长行驶, 对较长加工加较短行驶。存在争用时则不然, 因为通往快速机械臂的那条通道可能正是其他工件都在使用的通道, 而在其中产生的等待对矩阵是不可见的。此时较慢但连通性更好的机械臂可能是更优选择; 而在同一算例的不同拥堵水平下, 结论又可能反转。这一权衡要求两个要素同时在场: 异构的可选机械臂给了调度器一个替代目的地, 显式的通道网络使该替代方案具有不同的代价。去掉任何一个, 权衡即不复存在。

1.2 与最接近工作的差距

现有工作或者每次只处理这一耦合的一侧, 或者只在单一方向上把两侧连接起来。无冲突多机器人路由高度重视争用——终身式 (lifelong) 问题的表述甚至通过重塑边权把交通从瓶颈处引开 [10]——但任务及其释放时刻与端点均来自规划器之外, 规划器因而无权质疑产生这些任务的指派。拥堵感知的车队控制同样对拥堵路段施加惩罚, 却是在一组固定的运输订单上做派车, 且以局部车辆密度作为拥堵的代理度量 [3]。

在调度一侧, 这一组合已被形式化: 带无冲突运输约束的柔性作业车间调度问题把“具有位置相关加工时间的可选机器工序”与“无冲突车辆路由”耦合起来, 并由基于逻辑的 Benders 分解将其基准算例求解至可证最优 [8]。本文采用该问题类, 而非声称提出它。仍然敞开的是启发式区间——算例一旦超出精确分解可及的规模便进入该区间, 而在那里耦合仍是单向的。两阶段方案先固定排程、再针对它消解车辆冲突 [7], 因此第二阶段发现的争用无法回过头修改第一阶段。与本文最接近的工作加入了一个反馈步骤, 但正如其局限性小节自述, 它“主要作为两阶段顺序优化策略而非完全动态的实时闭环系统运作”, 依赖的是“一次性的反馈调整” [4]。该设计带来两个后果, 且都是结构性的而非偶然的: 反馈只调整时间轴而不修改任何指派; 并且它只施加于单个当前最优解一次, 而不施加于搜索所考察的每一个候选解。此外, 该工作是在每道工序仅一台可选机器、加工时间固定的作业车间基准上展开的, 上文那种改派权衡在其中根本无从出现。

概括起来, 两类最接近的工作各缺一半: 一类具备机器柔性却在常数运输时间矩阵上排产, 看不见拥堵; 另一类具备真正的无冲突路由却没有柔性, 指派由算例固定。本文开头那句“最短走廊拥堵时, 去一台更慢但连通性更好的机械臂可能更优”, 在它们的模型里连表述都无法完成。

1.3 本文方法

本文在两种意义上闭合这个回路, 并逐项量化每一种意义值多少。框架保留双层结构——上层决定机器指派与工序排序, 下层以时间窗预约生成无冲突路径——但沿两条通路连接两层。

评价通路把每一代中每一个候选解的适应度, 定义为下层真正为其规划过路径之后得到的完工时间。于是让行延误进入搜索所优化的目标本身, 而不再是事后的可行性校验。这一步的想法并不深刻, 深刻之处在于它很贵: 一次完整的无冲突解码比在理想最短路矩阵上解码昂贵一个数量级以上, 因此在固定算力下它必须与它所挤占的那些额外评价次数竞争。据我们所知, 此前没有工作量化过这笔交换是否划算; [4] 的作者本人正是把“将该机制扩展为完全迭代的回路”列为未来工作。

决策通路上承重的机制是预约表感知的车辆派遣。选车不再依据理想最短路对到达时刻的估计, 而是对预约表做一次两段式试探: 对每一个候选车辆, 依次询问“取货段”与“送货段”在当前预约状态下实际可行的最早完成时刻, 再取其中最优者。它所替换的恰是文献 [4] 正在使用的空闲优先规则, 因此这一机制针对的是现行的发

表实践, 而不是一个假想的弱基线。为使它在同挂钟预算下付得起, 本文给出两项可证保持最优选择的降本: 基于可采纳下界的剪枝, 以及胜者路径的复用。

我们同样实现并考察了更具雄心的第三条接口: 让下层不只返回时间, 而返回对关键链的归因——把关键链从工序序列扩展为工序与运输交替的链, 并给每一环标注五类成因之一 (工序、机器、上游、车辆、走廊), 其中最后一类指明被争用的路段及让行发生的时段; 再由两族算子据此行动: 改派把工序移到经由争用较轻的网络区域可达的机械臂 (改变地点), 错峰把被阻塞的工序提前或把阻塞者推后 (改变时间)。这一设计在直觉上比预约表试探更有吸引力, 但在同挂钟预算下其增益未获证实。本文如实报告这一点, 并把它与走廊定价、穷举式真解码一并作为受控负对照 (第 5 节), 用以说明为何在若干候选接口中最终选定了预约表感知派遣这一个。这些负对照不是本文的结论, 而是本文设计取舍的正当理由。

1.4 主要结果

为使“闭环”与“具体机制”两笔账互不冒领, 我们构造了一个四级基线阶梯, 其中前两级分别复现两篇已发表工作的结构: **B0** 为开环规划加真实执行, 对应 [4]; **B0+** 在冻结排产上只把车辆指派换成感知冲突的, 对应 [7] 的两阶段结构; **B1** 为闭环规划加规则派车; **B2** 为本文方法。这四级构成“是否闭环”与“派车方式”的 2×2 析因, 故每次只动一个因子的对比才可用于归因。所有方案——无论其计划从何而来——都必须放进同一个无冲突路由器执行并通过同一个独立校验器, 报告的都是可实现的完工时间。

结果显示两条主张各自成立。把拥堵搬进搜索目标 (固定派车方式) 使完工时间降低 18.86% ($p = 0.0000$), 这是闭环本身的价值; 在闭环内部把规则派车替换为预约表感知派遣 (信息完全对等, 只差派车一处决策) 再降低 2.45% ($p = 0.0054$), 这是该具体机制的价值; 端到端相对文献结构降低 21.35%, 94 胜 5 负 ($p = 0.0000$)。两项降本使派遣机制的加速比达到 2.33 倍、路由调用减少 47.4%, 单次评价的成本倍数从 12.2 压到 5.23, 这正是它在同挂钟预算下能够转为净收益的原因。

两条机制都显著, 但不可互相替代。开环下的试探派车值 4.59% ($p = 0.0000$), 远小于闭环的 18.86%; 端到端 21.35% 接近两者独立时应有的 22.6%, 呈近加性而非正交互。闭环内试探的平均增益 (2.45%, $p = 0.0054$) 被车队偏大两格上的反噬所拖低——适用上界是第 5.5 小节的主题。这也是我们坚持报告 2×2 析因而非一条消融链的原因: 只有分因子才能看见“主效应大、次效应有条件”。

稳健性方面, 上述增益在 10 个受控算例格上成立, 这些算例格沿三个因子族逐一单变——布局 (争用强度覆盖 6.6% 至 33.5%)、车臂比与柔性——因此可以分别读出增益随各因子的走势, 而不只是一个汇总均值。第 5.5 小节给出逐族结果: 闭环增益随争用强度大致上升; 试探派车在 $N_A/N_M = 0.5$ 最赚、在 1.0 与 2.0 两格反噬 (+4.8%, +4.7%), 划定适用上界; 柔性两档上试探增益接近, 说明它主要经派车与时序兑现, 并不强依赖指派余地。

1.5 实验协议

上述任何一条结论的成立都要求比通常做法更严格的实验协议, 原因是我们亲历过的三个陷阱。第一, 各机制的单次评价成本相差极大, 最贵与最便宜之间可达两个数量级, 因此在相同代数下比较等于把一个机制多消耗的算力也记作它的功劳; 改用同挂钟预算后, 若干表面上的增益会反转。第二, 在拥堵算例上完工时间的种子间离散度超过不同方案之间的差距, 单个种子几乎可以把各方案排成任意次序, 故所有比较均在多个随机种子上以配对 Wilcoxon 检验进行。第三个陷阱更隐蔽, 并且曾使我们得出过错误结论: 消融必须每次只改变一个因子, 而“每一行在上一行基础上增加一个机制”的链式消融只有在各行确实只差一个因子时才满足这一点。我们的早期设计并不满足, 而被混淆的那个对比会把一个机制报告为显著有益, 去混淆后的对比却报告它显著有害——同一批运行, 相反的符号, 两者都显著。

算例本身也是一个要设计的对象。集中在上下料出口处的拥堵会被所有工件在所有指派下穿越, 它只是抬高每一张排程的延误, 却不给任何机制可利用的余地。因此我们生成的算例以网络容量而非距离作为旋钮来调节

争用强度,并用一对仅在站口断面宽度上有差别的匹配算例,把“调度器可以绕开的拥堵”与“绕不开的拥堵”区分开来。

本文的贡献如下。

- (1) **问题层: 一个在最接近的两类工作中都无法表述的权衡。**机器柔性、加工效率异构与走廊拥堵三者同时在场,且拥堵进入目标函数。由此“最短走廊拥堵时,去一台更慢但连通性更好的机械臂可能更优”成为一个真实的决策;而在具备柔性但用常数运输时间矩阵排产的模型里 [7],以及在具备真正无冲突路由但机器由算例固定的模型里 [4],这句话都无法被表述。我们在既有的带无冲突运输约束的柔性作业车间调度问题类 [8] 内工作,采用该问题类而不声称提出它,并补充一个可控的争用强度刻画,使“何时值得管拥堵”成为可回答的问题。
- (2) **框架层: 闭环的价值被首次量化。**把无冲突路由放进适应度函数这一想法本身并不深刻,深刻之处在于它很贵,因而从未有人量过它值多少——[4]的作者本人正把完全迭代的回路列为未来工作。本文在同挂钟预算下给出这笔交换的汇率: 相对该文献的忠实结构,闭环使完工时间降低 18.86% ($p = 0.0000$),在预约表感知派遣下达到 17.75% ($p = 0.0000$)。贡献是“定量”加“让它付得起”,而不是这个念头本身。
- (3) **机制层: 预约表探测式派车,以及让它付得起的降本。**在闭环内部,该机制值 2.45% ($p = 0.0054$);它替换的正是文献 [4] 在用的空闲优先规则,因此打的是现行的发表实践。两项可证保持最优选择的优化——基于可采纳下界的剪枝与胜者路径复用——带来 2.33 倍加速、路由调用减少 47.4%,把单次评价的成本倍数从 12.2 压到 5.23。机制层面的解释是:AGV 之间是同构的,车与车唯一的差别在于站位与路是否通得过,因此选车是一个纯粹的拥堵决策,信息能够全额兑现;而选臂还掺入加工时长,后者往往压过走廊差异。
- (4) **角色分工与适用上界。**闭环是主效应;预约表感知派遣是有车队规模上界的次效应。两者在开环与闭环下都显著,端到端接近独立乘积 (近加性),但开环试探单独远不能代替闭环;大车队两格上闭环内试探反而有害 (+4.8%, +4.7%)。这一条把“机制有效”收束为“在什么条件下有效”。
- (5) **方法层: 一套把机制与算力分开的实验装置。**包括争用强度可控的算例生成 (标定至 6.6% 至 33.5%)、统一执行器 (任何来源的计划都经同一个仿真器与同一个独立校验器)、同挂钟而非同代数的预算口径,以及四级基线阶梯且每一级对应一篇真实发表的结构。我们并附三个受控负对照与一条代价律:走廊定价在同等算力下无益,凭证制打的得分只捕获随机与神谕之间的一小部分,穷举式真解码在同挂钟下反而落后;而试探档的代数与车队规模之积近似为常数,车队足够大时剪枝失效。这些负对照不是本文的结论,而是“我们为何选定这个设计”的正当理由。

1.6 论文组织

本文余下部分组织如下。第 2 节回顾相关工作,包括考虑运输的柔性作业车间调度、任务外生的无冲突路由,以及两者的一体化表述,并定位本文的缺口。第 3 节给出问题描述与数学模型,含符号表与编号的建模假设。第 4 节详述闭环双层框架: 两条反馈通路、下层的时间窗预约路由、上层的编码与事件驱动解码、预约表感知的车辆派遣及其两项降本,并给出复杂度与代价分析。第 5 节报告实验协议与结果: 四级基线阶梯、主结果、主效应与不可替代性、稳健性与适用上界、消融与受控负对照,以及案例的关键链归因。第 6 节总结全文并讨论局限与后续方向。

全文符号汇总于表 2。

2 相关工作

本文的问题位于两支长期各自独立发展的文献之间: 考虑运输的柔性作业车间调度, 它决定什么在何处以何种次序被加工, 却把车辆抽象掉; 以及无冲突多机器人路由, 它在时空上消解车辆争用, 但任务来自别处。下面依次回顾两者, 再回顾数量较少的一体化工作, 并在最后定位本文的缺口。

2.1 考虑运输的柔性作业车间调度

含加工机器与运输车辆的柔性作业车间调度问题 (FJSP_PT) 通过把物料移动纳入排程来扩展 FJSP。Xin 等 [9] 系统综述了该领域至 2023 年的进展, 梳理其假设、约束、目标与基准算例, 并把求解方法分为精确、启发式、元启发式与群智能几类。其中占主导地位的建模约定是一个按位置对索引的常数行驶时间矩阵: 车辆把工件从 a 移到 b 需要固定的 $t(a, b)$, 与车间里同时发生的其他事情无关。于是车辆只是一种能力资源——它们为工件而竞争, 却从不为空间而竞争。

这一传统在组合优化一侧进展迅速。Meng 等 [5] 给出多 AGV 情形下 FJSP 的约束规划模型, 关闭了 29 个此前开放的基准算例并改进了 27 个最优已知解, 并与一个由约束规划辅助的双种群遗传算法相配合。Qin 等 [6] 把质量-多样性存档与 Q 学习的区域选择结合, 并设计了由 AGV 与关键路径共同制导的局部搜索。Fu 等 [1] 用改进遗传算法在柔性装配车间中一体化地决策生产与物流。但在上述全部工作中, 运输时长都是从矩阵中读出的, 因此一辆车永远不会被另一辆车延误。

综述本身明确指出这是一个局限而非已成定论的建模选择, 它观察到“目前多数调度问题的假设过多”, 而这些假设“会降低问题复杂度, 甚至改变问题的性质, 例如当运输车辆数量足够多时” [9]; 并进一步指出, 工件到达机器的时刻在实践中受交通拥堵影响, 而加工-运输两阶段流动中隐含的依赖关系必须先被处理, 一张排程才谈得上可行, 更不必说鲁棒。

2.2 任务外生的无冲突路由与拥堵感知派车

互补的另一支文献把争用本身当作研究对象。在多智能体路径规划、特别是其终身式变体中, 多个智能体必须在共享图上无碰撞地行进, 而拥堵被公认会在远未达到容量上限时就已损害吞吐。Zhang 等 [10] 的做法是优化一张制导图, 用其边权把智能体从瓶颈处引开并抑制迎面运动, 从而提升若干终身式求解器的吞吐。在制造场景中, Kim 与 Kim [3] 为大规模 AGV 车队提出拥堵函数、拥堵惩罚与拥堵感知的派车规则, 并在离散事件仿真中加以评估。

这支文献有两个特征对本文尤为重要。第一, 任务是外生的: 其释放时刻、起点与终点都是交给路由器的输入, 路由器因而从来没有资格质疑产生这些任务的那个指派。第二, 凡对拥堵作量化之处, 所用的通常是一个代理量——[3] 中的局部车辆密度、[10] 中离线学到的边权——而不是某一次具体的预约实际给某一个具体任务造成的延误。这一区别决定了反馈能携带什么: 代理量只能说明“此处大体拥挤”, 而具体的让行事件才能指名“哪一段、哪个时段、被谁先占”, 后者才是上层据以决定“是把工序换个地方, 还是仅仅换个时间”所需的信息。

2.3 调度与无冲突路由的一体化: 反馈深度与本文定位

把两者耦合起来的工作数量较少。在精确求解一侧, van Os 与 Basten [8] 形式化了带无冲突运输约束的柔性作业车间调度问题, 它把“具有位置相关加工时间的可选机器工序”与“无冲突车辆路由”结合起来, 并以基于逻辑的 Benders 分解求解: 约束规划主问题给出排程, 整数规划子问题为车辆规划路径并返回由冲突驱动的割。多数基准算例被求解至可证最优, 其中约一半相对此前的启发式方法把完工时间改进了至少 10%。该方法受限于精确分解可及的算例规模, 其作者亦指出, 更积极地利用“关于不可行性成因的信息”是自然的下一步。本文采用这

表 1. 与最接近的几支工作的定位对比。”机器异构”指一道工序有多台可选机器且加工时间随机器而变;”耦合频次”指路由结果以何种频次、沿何种方向到达调度决策;最后一列是穿过层间接口的内容。本文实现了实现时间、冲突凭证与走廊价格三类接口,并按各自的代价与收益逐一报告,其中承重的是预约表试探。

工作类别	文献	机器异构	车辆争用	耦合频次	反馈内容
FJSP_PT 元启发式	[1, 5, 6]	有	常数矩阵	—	—
终身式 MAPF	[10]	—	无冲突	任务外生	边权
拥堵感知派车	[3]	无	密度代理量	单向	拥堵惩罚
两阶段 FJSP + 路由	[7]	有	无冲突	单向	无
强化学习调度 + PBS	[4]	无	无冲突	一次性修正	到达时刻
精确 LBBD	[8]	有	无冲突	主-子问题	冲突驱动割
本文		有	无冲突	每个候选解	实现时间 + 预约表试探

一问题类,并把上述”由不可行成因反馈到可行行动决策”的思路移到启发式区间;第 4 节的预约表试探正可视为该思路在启发式框架中的对应物。

在启发式一侧,耦合仍是单向的。Sun 等 [7] 先用混合遗传算法排产,再在第二阶段用 AGV 编码的遗传算法与时间窗 Dijkstra 消解 AGV 冲突;第二阶段发现的争用无法回过头修改第一阶段。Li 等 [4] 最接近一个回路:其调度层用强化学习,任务分配由甘特图导出,再由基于优先级搜索的路径层把实现的运输时长反馈回时间轴。该文自己的局限性小节把结果描述为”主要作为两阶段顺序优化策略而非完全动态的实时闭环系统”运作,依赖”一次性的反馈调整”,并指出”最终解的全局最优性部分依赖于初始排程的质量”。其反馈只携带时间,因而只能沿时间轴平移排程而无法修改任何指派;并且其基准是每道工序仅一台可选机器的经典作业车间算例,改派在其中根本不是一个决策。

本文的缺口正落在”闭环深度”上,而不落在”是否考虑冲突”上。上述两项工作都已认真对待冲突;区别在于路由结果到达调度决策的频次与内容:是每一代对每一个候选解都到达,还是只对单个当前最优解到达一次;是只携带时间,还是携带足以支持改派的信息。

关键链推理在作业车间局部搜索中早已成熟,近期的 FJSP_PT 工作也在考虑运输的前提下加以运用 [6]。当路由变为无冲突之后发生变化的是诊断的内容。基于行驶时间矩阵算出的关键链,可以把一次迟到的开工归因于缓慢的前驱或繁忙的机器,却无法把它归因于某条走廊,因为在那个模型里走廊并不作为被争用的资源而存在。一旦占用被按时间窗预约,一次迟到的开工便可被追溯到具体的路段、具体的时段,以及先行预约它的那个任务。

对算例设计,[9] 亦指出该领域的算例”主要在规模上有差别,缺少针对关键问题参数取值的专门设计”,并特别把平均运输时间与平均加工时间之比 $\bar{T}_t/\bar{T}_p$ 列为极可能影响算法表现的量,呼吁扩充测试集。本文在此之上补充一个只有在路由被显式建模后才可见的参数:一个算例的拥堵中有多少是与指派相关的。集中在上下料出口的拥堵会被所有工件在所有指派下穿越,因而不奖励任何决策;而只服务于部分机器的走廊上的拥堵则可以被绕开。两个 $\bar{T}_t/\bar{T}_p$ 完全相同的算例,留给调度机制的可操作余地可能完全不同。本文据此生成争用强度可控的算例(第 5 节),并对实验协议提出一项要求:当各机制的单次评价成本相差达两个数量级时,在相同迭代次数下比较所测得的并非其所声称的东西,且误差方向并不随机——它偏向每次迭代做更多工作的那一个,而那通常正是被提出的方法。

表 1 汇总了上述比较。本文工作的问题类并不新,它就是 [8] 所形式化的那一个,本文采用它;敞开的是启发式区间——算例在此超出精确分解可及的规模,而调度与路由之间的耦合在此仍然或缺失、或单向、或只对单个当前最优解施加一次。本文的贡献是在这个区间内给出一个闭合到每一个候选解的双层框架,指认其中真正承重的那一个层间机制(预约表感知的车辆派遣),用可证等价的降本使它在同等算力下付得起,并量化它相对闭环主效应的角色与适用上界。

表 2. 符号说明。

符号	含义
$J = \{1, \dots, n\}, O_{ji}$	工件集合; 工件 j 的第 i 道工序, $i \leq n_j$
$M = \{1, \dots, N_M\}, \Omega_{ji} \subseteq M$	机械臂集合; O_{ji} 的可选机械臂集
t_{jim}^P	O_{ji} 在机械臂 $m \in \Omega_{ji}$ 上的加工时间
$K = \{1, \dots, N_A\}$	同构的单载量车辆
$G = (V, E), \tau_e$	车间图; 走廊 $e \in E$ 的通行时间
$v_0, \text{loc}(m)$	上下料站; 机械臂 m 所在节点
$t^*(a, b)$	G 中 a 到 b 的最短路时间, 不计争用
$\delta_{\text{ret}} \in \{0, 1\}$	完工工件是否返回 v_0 (基准取 1)
S_{ji}, C_{ji}, A_{ji}	O_{ji} 的开工、完工与物料到达时刻
$C_{\max}$	完工时间

3 问题描述与数学模型

3.1 问题描述

一个算例由 n 个工件、 N_M 台机械臂与 N_A 辆自动导引车构成, 它们在一个以图表示的车间中运行。工件 j 具有固定的 n_j 道工序链 $O_{j1} \rightarrow \dots \rightarrow O_{jn_j}$; 工序 O_{ji} 可在其可选集 $\Omega_{ji} \subseteq M$ 中的任一机械臂上加工, 在机械臂 m 上耗时 t_{jim}^P , 该值取决于所选的机械臂。机械臂占据图中的固定节点, 工件由车队在这些节点之间搬运。求解一个算例意味着同时确定四组决策:

- (i) 把每道工序 O_{ji} 指派给某台机械臂 $m \in \Omega_{ji}$;
- (ii) 为每台机械臂上被指派的工序排序;
- (iii) 把由此派生的运输任务指派给车辆, 并为每辆车的任务排序;
- (iv) 为每一次车辆移动在图上规划路径, 要求在时间与空间上无冲突, 允许等待。

目标是最小化完工时间 $C_{\max}$ 。全文符号汇总于表 2。

3.2 建模假设

- A1. 静态算例与完全信息。** 全部 n 个工件在 $t = 0$ 时刻均已位于 v_0 , 不设释放时刻与交货期。加工数据、布局 G 与车辆状态在规划时均为已知。
- A2. 工件。** 每个工件具有固定的前后约束链。加工与运输均不可抢占, 工件不可拆分, 不同工件之间不存在在外生的先后约束。
- A3. 机械臂: 部分柔性且工时异构。** 每道工序 O_{ji} 可在任一 $m \in \Omega_{ji}$ 上加工, 多数工序满足 $|\Omega_{ji}| \geq 2$, 完全柔性 $\Omega_{ji} = M$ 是其特例。加工时间在无关机器 (unrelated) 意义上与机器有关: 当 $m \neq m'$ 时, t_{jim}^P 与 $t_{jim'}^P$ 之间不必存在任何关系。一台机械臂同时只加工一道工序, 在计划期内不发生故障, 与顺序相关的准备时间被并入 t_{jim}^P 。
- A4. 缓冲区与装卸。** 每台机械臂的输入、输出缓冲区以及 v_0 处的存储容量充裕, 因而不发生阻塞。装载与卸载时间可忽略, 且不占用机械臂。
- A5. 车辆。** N_A 辆车同构、单载量, 空载与满载均以恒定速度行驶, 不建模加减速与电量。一个运输任务是一次单程移动——从 v_0 到第一台机械臂、在前后两台不同机械臂之间, 或当 $\delta_{\text{ret}} = 1$ 时从最后一台机械臂返回 v_0 ; 任一车辆可执行任一任务, 因此同一工件的连续移动可以在不同车辆之间接力。卸载之后车辆不被强制返回 v_0 , 而是按其自身的任务链重新占位, 且其出发时刻可由路由层推迟。

A6. 路网与无冲突性。 G 强连通, 各走廊的通行时间 τ_e 为已知常数。走廊是排他资源: e 的两个方向共享同一份预约, 因而任一时刻最多一辆车占用 e , 这同时排除了迎面冲突与同向超车。占用区间取半开形式 $[t, t + \tau_e)$, 故第二辆车可以恰好在 $t + \tau_e$ 进入。节点上设有容量充裕的停靠位, 它与通行资源相互独立; 因此等待发生在停靠位上, 并且总是被允许的。

A7. 构造性的无死锁。 路径以走廊资源上的时间窗预约方式规划, 新路径只能占用空闲的时间窗。结合 A4 与 A6, 这使得任何指派、排序与任务链都可被解码为一张无冲突、无死锁且完工时间有限的排程, 因而永远不需要对不可行解做修复。

A8. 不在建模范围内。 机器与车辆故障、有限缓冲与阻塞、电池充电, 以及工件的动态到达, 均留待后续工作。

其中 A1、A2、A4 与 A5 在 FJSP_PT 文献中是标准假设 [9]。真正塑造本文的是 A3(部分柔性机械臂上与机器相关的加工时间) 与 A6(把争用表述为显式的排他资源约束, 而不是一个常数行驶时间矩阵), 以及由此导出的一个事实: 运输时长是路由决策的输出, 而不是输入参数。

3.3 决策变量与派生的运输任务

令 $x_{jim} \in \{0, 1\}$ 表示 O_{ji} 被指派给机械臂 m , 记 $m(j, i)$ 为如此选定的机械臂, $d_{ji} = \text{loc}(m(j, i))$ 为其所在节点。当 $\delta_{\text{ret}} = 1$ 时, 为每个工件追加一道位于 v_0 、加工时间为零的伪工序 O_{j, n_j+1} , 用以表示完工工件的返回; 这样可使完工时间的定义保持统一。

与车辆路径问题不同, 运输任务并不是输入的一部分, 而是由指派派生出来的:

$$\mathcal{T}(x) = \{(j, i) : d_{j, i-1} \neq d_{ji}\}, \quad d_{j0} \equiv v_0, \quad (1)$$

其中任务 (j, i) 把工件 j 从 $d_{j, i-1}$ 搬到 d_{ji} , 并在 $r_{ji} = C_{j, i-1}$ 时刻就绪 (约定 $C_{j0} = 0$)。若前后两道工序被放在同一台机械臂上, 则不产生任何任务。因此, 运输任务的数量与端点都是决策组 (i) 的后果: 一个把工件始终留在同一台机械臂上的指派只需搬运两次, 而一个每道工序都换臂的指派则需搬运 $n_j + 1$ 次。任务到车辆的指派用 $y_{tk} \in \{0, 1\}$ 表示, 对每个 $t \in \mathcal{T}(x)$ 有 $\sum_{k \in K} y_{tk} = 1$, 且分配给同一辆车的任务是全序的。

这一点值得强调: 在常数行驶时间矩阵的模型中, 运输量是给定的; 而在本文的模型中, 上层的指派决策同时决定了要搬多少次、从哪搬到哪, 以及这些搬运会在哪些走廊上彼此争用。

3.4 时序关系与可行性条件

记 $p_{ji} = \sum_{m \in \Omega_{ji}} x_{jim} t_{jim}^P$ 为实现的加工时间。指派、前后约束与机械臂的单件加工能力给出

$$\sum_{m \in \Omega_{ji}} x_{jim} = 1, \quad \forall j, i, \quad (2)$$

$$C_{ji} = S_{ji} + p_{ji}, \quad \forall j, i, \quad (3)$$

$$S_{ji} \geq A_{ji}, \quad \forall j, i, \quad (4)$$

$$S_{j'i'} \geq C_{ji} \vee S_{ji} \geq C_{j'i'} \quad \forall (j, i) \neq (j', i') \text{ 位于同一机械臂}, \quad (5)$$

其中 A_{ji} 是工件实际出现在 d_{ji} 的时刻。若 $(j, i) \notin \mathcal{T}(x)$, 则工件本已在该处, $A_{ji} = C_{j, i-1}$ 。否则 A_{ji} 由承运任务 (j, i) 的那辆车产生: 记 a_{ji}^E 为该车经空载段到达取货节点的时刻, 则

$$\ell_{ji} = \max(a_{ji}^E, r_{ji}), \quad A_{ji} = \text{于 } \ell_{ji} \text{ 出发的满载段的到达时刻}, \quad (6)$$

即车辆可以等工件, 工件也可以等车辆。无论空载段还是满载段, 每一段都是 G 中的一条通路 $(e_1, \dots, e_q)$, 其走廊进入时刻 $\theta_1, \dots, \theta_q$ 满足

$$\theta_1 \geq \sigma, \quad \theta_{l+1} \geq \theta_l + \tau_{e_l}, \quad l = 1, \dots, q-1, \quad (7)$$

其中 σ 是该段可以开始的时刻; 上述不等式取严格不等号的时刻恰好就是车辆在停靠位等待的时刻, 而该段在 $\theta_q + \tau_{e_q}$ 结束。最后是争用约束:

$$[\theta, \theta + \tau_e) \cap [\theta', \theta' + \tau_e) = \emptyset \quad (8)$$

该式对同一走廊 e 的任意两次不同占用都成立, 与方向无关, 也与涉及哪些车辆无关。目标函数为

$$\min C_{\max}, \quad C_{\max} = \max_{j \in J} C_{j, n_j + \delta_{\text{ret}}}. \quad (9)$$

有两个推论值得记录, 因为第 4 节的算法正建立在它们之上。第一, 在任何半活动排程中, (4) 与 (5) 可收紧为

$$S_{ji} = \max(A_{ji}, F_{m(j,i)}), \quad (10)$$

其中 F_m 是机械臂 m 上排在 O_{ji} 之前那道工序的完工时刻。于是开工时刻恰由两种成因之一支配, 且两者中哪一个取到最大值是良定义的; 正是这一点使得后文把一次延误归因于某个成因成为有意义的判定, 而不是启发式的猜测。第二, (8) 是模型中唯一通过空间而非通过共享机器来耦合不同工件的约束。

3.5 与 FJSP_PT 的关系

删去 (8) 后各段彼此独立, 于是每一段都可以走最短路, 且 (7) 处处取等号。运输时长随之退化为常数矩阵 $t^*(a, b)$, 模型也就变成标准的考虑运输的柔性作业车间调度问题。因此两个问题之间的全部差别只是一族约束——以及随之而来的一个事实: 一次移动的时长不再是参数, 而是由所有车辆的路径共同决定的变量。同一松弛还提供了自然的乐观估计 t^* , 它在后文既用作下界, 也在开环基线中充当派车器对到达时刻的估计。

3.6 计算复杂度

命题 1. 由 (2)–(9) 定义的问题是强 NP 难的; 且当所有工序均满足 $\Omega_{ji} = M$ 时依然如此。

证明. 限制到如下算例: 对所有 $e \in E$ 有 $\tau_e = 0$, 每台机械臂各由一条专用走廊与 v_0 相连, 且 $N_A = n$ 。此时每一段行驶耗时为零、占用的半开区间 $[t, t) = \emptyset$, 故 (8) 恒真, 任何车辆都无法延误另一辆车; 又因每个工件独占一辆车, (6) 从不起作用, $A_{ji} = C_{j,i-1}$ 。余下的正是取 $A_{ji} = C_{j,i-1}$ 的 (2)–(5), 也就是无关机器情形下的柔性作业车间调度问题。它的两个特例已是强 NP 难的: 处处取 $|\Omega_{ji}| = 1$ 时退化为经典作业车间问题, 在三台机器下即为强 NP 难 [2]; 取 $\Omega_{ji} = M$ 且 $n_j = 1$ 时退化为无关并行机上的完工时间最小化问题, 它包含同型并行机调度, 在机器数可变时为强 NP 难。上述每个限制均为多项式规约, 故一般问题是强 NP 难的; 第二个特例同时说明完全柔性并不会使问题变容易。 $\square$

该规约刻意关闭了争用, 这表明难度仅从调度一侧就已继承而来; 路由一侧是在此之上叠加难度, 而非取代它——因为把决策 (i)–(iii) 冻结之后得到的路由子问题本身就是一个无冲突多智能体路由问题, 且其中各任务的释放时刻与端点是彼此链接的。小规模算例上存在精确方法: van Os 与 Basten [8] 用基于逻辑的 Benders 分解关闭了同一问题类的多数算例; 本文引用它们是为了给本文所针对的启发式区间划定边界, 而不是与之竞争。

3.7 算例特征的刻画

由于本文所研究机制的效果取决于算例的结构而非规模, 每个算例都记录如下描述量。前四个是常规的; [9] 的综述特别指出其中第一个极可能左右算法的表现。

- $\bar{T}_t/\bar{T}_p$: 机械臂节点之间的平均最短路行驶时间除以平均加工时间, 即搬运相对于加工的权重。
- 异构度 $H: \{t_{jim}^P : m \in \Omega_{ji}\}$ 的变异系数在所有工序上的均值。 $H = 0$ 意味着可选机械臂在工时上可以互换, 改派权衡随之关闭。
- 柔性度 $F = \text{mean}_{ji} |\Omega_{ji}|/N_M$: 改派可以周旋的余地。
- N_A/N_M : 瓶颈更可能位于运输一侧还是加工一侧。

其余描述量是本文场景特有的, 它们存在的理由是: 拥堵本身并不是恰当的度量, 真正要紧的是其中有多少是调度决策可以避开的。令 $\text{cut}(v_0)$ 为使 v_0 与所有机械臂断开所需删除的走廊的最小基数集合。

- 漏斗占比 $\varphi \in [0, 1]$: 一次典型的“ $v_0 \rightarrow$ 机械臂”行程中, 花在所有最短 $v_0 \rightarrow$ 机械臂路径都共用的走廊上的时间比例。这类走廊在任何指派下都会被每个工件穿越, 因而它们造成的延误对决策 (i)–(iii) 不变; φ 越大, 任何反馈机制可利用的信号被稀释得越厉害。
- $|\text{cut}(v_0)|$, 即漏斗宽度, 它给出可同时离站的车辆数上界; 取值为 1 表示单点漏斗。
- 远组割: 把到 v_0 的距离超过中位数的那些机械臂隔离出来所需的最小走廊数。只割单台机械臂是没有信息量的, 因为它自己的支路总给出 1; 组割衡量的是指派决策真正争夺的那个深层瓶颈的容量。

最后, 每个算例都附带一个零成本的复合下界 $\text{LB} = \max(\text{LB}_{\text{chain}}, \text{LB}_{\text{load}}, \text{LB}_{\text{cut}})$, 其三个分量分别松弛模型的不同部分, 且互不占优。 LB_{chain} 逐工件取“最短入场行程 + $\sum_i \min_{m \in \Omega_{ji}} t_{jim}^P$ + 最短返回行程”, 即松弛掉臂间移动与全部排队。 LB_{load} 取 $\sum_{j,i} \min_m t_{jim}^P / N_M$, 即松弛前后约束与运输。 LB_{cut} 利用漏斗结构: 当 $\delta_{\text{ret}} = 1$ 时每个工件穿越 $\text{cut}(v_0)$ 两次, 把 $X = 2n$ 次穿越分配到割集上以最小化最繁忙走廊的负载, 得到

$$C_{\max} \geq X / \sum_{e \in \text{cut}(v_0)} \tau_e^{-1}. \quad (11)$$

该下界是松的——与最优已知解之间实测的间隙大致在 35% 到 61% 之间——因此它用于数量级核查与回归防护, 而不作为解质量的证书。

4 闭环双层框架

4.1 框架总览: 两条反馈通路

本框架把第 3 节的四组决策分配到两层。上层用一个 memetic 算法在指派与排序上搜索; 下层以时间窗预约把给定的一组运输任务转化为无冲突路径。使它区别于两阶段方案或一次性修正方案的, 不是这种分层, 而是什么东西穿过分层界面、沿哪个方向穿过、以及穿过的频次。

评价通路。 每一代中每一个个体的适应度, 都是下层把它的全部运输任务无冲突地规划完毕之后得到的完工时间, 其中已包含让行延误。上层从不看到乐观估计, 因此从不优化一个代理目标。

决策通路。 下层的预约表是上层做车辆派遣决策时可以查询的对象, 而不只是事后被告知结果。承重的机制是预约表感知的车辆派遣: 选车之前先对预约表做一次两段式试探, 用实际可达的送达时刻代替理想最短路的估计。这消去了框架中最后一处开环残余。

评价通路使目标函数诚实, 决策通路使派车决策不再依赖估计。两条通路的价值在第 5 节被分别量化: 闭环是主效应, 预约表感知派遣是有车队规模上界的次效应; 二者近加性叠加, 开环试探单独远不能代替闭环。

除派遣之外, 我们还实现了另外两种更具雄心的层间接口——为走廊时段定价, 以及回传结构化的冲突凭证以制导局部搜索。它们在同挂钟预算下未能转为净收益, 故本章仍完整描述其构造 (第 4.4.1、4.6 小节), 而把度量结果放在第 5 节作为受控负对照。这样安排的理由是: 一个构造上站得住、却仍不划算的机制, 比只报告成功的那个更有信息量——会自己设计出它的读者可以直接看到它的代价。

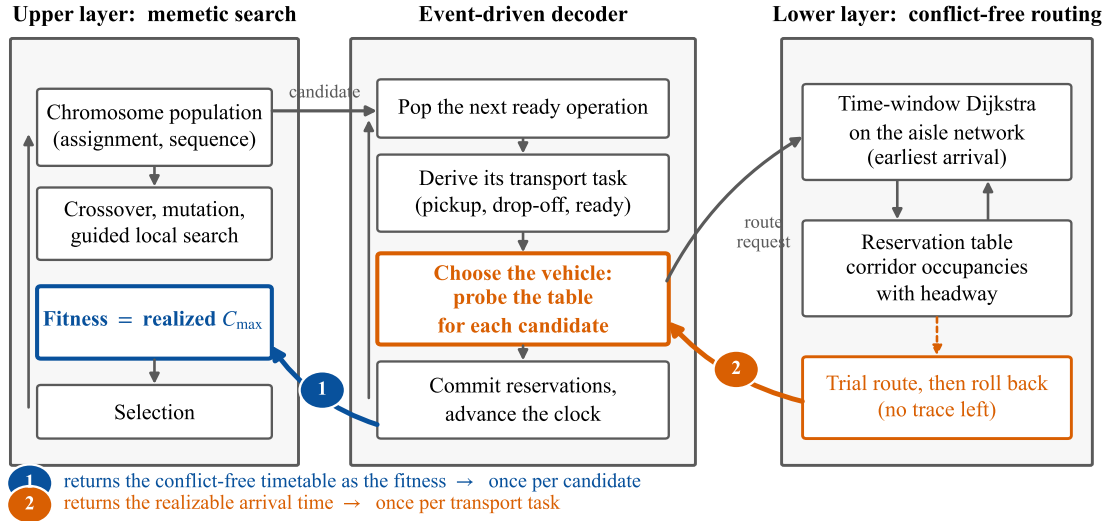


Fig. 1. 闭环双层框架。上层以 memetic 算法搜索指派与排序, 下层以时间窗预约做无冲突路由, 中间的事件驱动解码器把上层的一个候选解翻译成一串运输任务。两条反馈通路以颜色区分且进入上层的位置不同: 通路一 (蓝, 标记 1) 把下层完成的无冲突时间表回传为适应度, 每个候选解触发一次; 通路二 (橙, 标记 2) 在选车之前查询预约表、取回实际可达的送达时刻, 每个运输任务触发一次, 试探完毕即回滚, 不在表上留下痕迹。第 5 节的四级基线阶梯所变化的正是这两条通路的有无。

框架的整体结构见图 1。图中特意让两条反馈通路进入上层的位置不同, 因为这正是它们代价不同的根源: 通路一每个候选解触发一次, 通路二每个运输任务触发一次。

4.2 下层: 时间窗预约与无冲突路由

4.2.1 预约表。 唯一被争用的资源是走廊 (假设 A6), 因此预约表把每条走廊映射到一个按时间排序的半开占用区间列表 $[t, t + \tau_e)$, 每个区间标注占用它的车辆与任务。该表支持四种操作: 查询不早于给定时刻的最早可行进入时刻、提交一次占用、释放某个任务的全部占用, 以及对整表做检查点与回滚。节点不被预约: 穿越节点耗时为零, 而等待总是发生在停靠位上, 假设 A6 已把停靠位与通行资源分离。因此只需一种资源类型, 这使路由器与那个独立的可行性校验器都保持简洁。

4.2.2 时间窗 Dijkstra。 一段行驶用标签设定 (label-setting) 搜索来规划, 其中标签 (u, t) 表示“在时刻 t 停在节点 u 的停靠位上”, 其代价为 t (算法 1)。松弛一条外出走廊时向预约表询问最早进入时刻 $t' \geq t$, 等待恰恰在此处产生: 间隔 $t' - t$ 就是在 u 的停靠位上耗掉的时间, 而 (7) 中的松弛量由搜索自身生成, 不需要另行建模。在一次解码之内已提交的路径不会被撤销, 因此规划顺序就是解码器生成任务的顺序——即带有确定性优先级的优先级规划。

引理 1 (最早到达即充分)。在每个节点上只保留最早到达时刻, 即可得到相对当前预约表而言到达时刻最小的行驶段。

证明。停靠位容量充裕且不占用通行资源, 因此一辆自时刻 t 起停在 u 的车, 只需一直等到 t' , 便可复现任何在 $t' > t$ 到达 u 的车所能采取的行为。于是到达时刻构成一个占优判据: 更早的标签绝不会更差。又因每次走廊松弛关于 t 单调不减, 该搜索是标签设定式的, 目标节点第一次被取出时即为最优。 $\square$

算法 1 ROUTE: 无冲突的单段路径规划

Require: 起点 s 、终点 g 、最早出发时刻 t_0 、预约表 R 、是否提交标志 $commit$

Ensure: 到达时刻, 以及走廊占用区间的序列

```

1: 将标签  $(s, t_0)$  压入以时间为键的优先队列
2: while 队列非空 do
3:   弹出时间最小的标签  $(u, t)$ 
4:   if  $u = g$  then
5:     回溯得到各占用区间; 若  $commit$  为真则逐一写入  $R$ ; return 到达时刻  $t$ 
6:   end if
7:   for all 自  $u$  外出的走廊  $e = (u, v)$  do
8:      $t_{in} \leftarrow R.EARLIESTENTRY(e, t, \tau_e)$  ▷ 在  $u$  的停靠位等到  $t_{in}$ 
9:     松弛标签  $(v, t_{in} + \tau_e)$ 
10:  end for
11: end while

```

正是该引理使我们可以使用普通的 Dijkstra 标签, 而不必采用安全区间规划的区间标签。它的脆弱之处很有启发性: 一旦引入第二个判据 (例如为占用支付的价格), 占优关系便不再是全序, 从而被迫改用多标签 Pareto 搜索。第 4.4.1 小节会回到这一点, 而它正是定价机制在同等算力下失效的技术根源。

由于预约的数量有限、时间上有界, 且 G 强连通, 路径总是存在的: 最坏情况下车辆一直等到所有既有占用全部过期, 然后畅通行驶。这一观察使下文的命题 2 成立, 也是无需死锁修复的原因。单次调用的代价为 $O(|E|W \log |V|)$, 其中 W 为每条走廊上预约区间数的均值。

对空表在所有取送节点上运行同一搜索, 即得到第 3 节的无争用矩阵 t^* 。它在三处被用到: 开环配置中派车器的估计值、改派算子中的邻近项, 以及两阶段基线的运输时间。它同时是本章第 4.5 小节剪枝所依赖的可采纳下界。

4.3 上层: 编码与事件驱动解码

4.3.1 染色体编码. 设工序总数为 N , 一个个体由两段构成: 长度为 N 的机器指派向量, 其第 k 个分量在固定枚举下为第 k 道工序取自 Ω_{ji} 的一台机械臂; 以及工序序列, 它是一个可重排列, 其中工件 j 出现 n_j 次, 第 x 次出现代表 O_{jx} 。该序列是一个全局优先级次序, 共用同一台机械臂的工序之间的相对先后由它读出。

返回行程被折叠进同一表示。当 $\delta_{ret} = 1$ 时, 每个工件额外贡献一次出现, 对应第 3 节的伪工序, 其机械臂固定为 v_0 、加工时间为零。于是返回移动与普通移动在同一条流中竞争车辆, 并由同一段基因排序, 解码器中不需要任何特例分支。

任务到车辆的指派有意不编码, 而是在解码过程中决定 (第 4.4 小节)。文献中存在为它设置第三段基因的做法, 但那会扩大搜索空间, 并且放弃“任何染色体都可解码”这一保证, 从而不得不引入修复算子。

4.3.2 事件驱动解码器. 解码器 (算法 2) 就是评价通路。它自左向右扫过工序序列, 为每台机械臂维护其空闲时刻, 为每个工件维护其当前节点与就绪时刻, 为每辆车维护其当前节点与可用时刻。当某道工序的机械臂与工件当前位置不同时, 便生成 (1) 的运输任务, 选定一辆车, 并针对实时的预约表规划两段路径: 到取货点的空载段与到目的地的满载段, 两者由 (6) 衔接。随后工序按 (10) 开工, 解码器同时记录两项中哪一项取到了最大值——即 `bind` 字段, 后文全部归因都以它为依据。

由此得到两条结构性性质。其一, 预约是按扫描生成任务的次序做出的, 因此一个染色体总是产出唯一一张时间表: 评价是确定性的, 这正是第 5 节同预算协议所依赖的前提。其二, 状态严格向前流动, 因此一个只在位置 x 之后扰动序列的邻居, 会使 x 之前的每个事件与每次预约保持不变——这是增量式评价的基础; 我们把它记为一项可用但当前实现未利用的加速手段。

算法 2 DECODE: 以完整无冲突仿真作为适应度

Require: 算例、路网、指派 x 、工序序列

Ensure: 完工时间、时间表、逐走廊的让行统计量

```

1: 对所有机械臂置  $F_m \leftarrow 0$ ; 对所有工件置  $\text{pos}_j \leftarrow v_0, \text{ready}_j \leftarrow 0$ 
2: 对所有车辆置  $\text{loc}_k \leftarrow v_0, \text{avail}_k \leftarrow 0$ ; 清空预约表
3: for all 按序列次序取出的工序  $O_{ji}$  (含伪工序) do
4:    $m \leftarrow$  在  $x$  下  $O_{ji}$  的机械臂;  $d \leftarrow \text{loc}(m)$ 
5:   if  $\text{pos}_j = d$  then
6:      $A_{ji} \leftarrow \text{ready}_j$  ▷ 同臂连续, 不生成运输任务
7:   else
8:     选定车辆  $k$  (第 4.4 小节)
9:      $a^E \leftarrow \text{ROUTE}(\text{loc}_k, \text{pos}_j, \text{avail}_k)$ ;  $\ell \leftarrow \max(a^E, \text{ready}_j)$ 
10:     $A_{ji} \leftarrow \text{ROUTE}(\text{pos}_j, d, \ell)$ ;  $\text{loc}_k \leftarrow d$ ;  $\text{avail}_k \leftarrow A_{ji}$ 
11:    按走廊累计让行时间
12:   end if
13:   若  $F_m > A_{ji}$  则  $\text{bind} \leftarrow \text{machine}$ , 否则  $\text{bind} \leftarrow \text{arrive}$ 
14:    $S_{ji} \leftarrow \max(A_{ji}, F_m)$ ;  $C_{ji} \leftarrow S_{ji} + p_{ji}$ ;  $F_m \leftarrow C_{ji}$ ;  $\text{pos}_j \leftarrow d$ ;  $\text{ready}_j \leftarrow C_{ji}$ 
15: end for
16: return  $C_{\max} = \max_{ji} C_{ji}$  与时间表
```

命题 2 (任何染色体都可解码). 对任意指派与任意工序序列, 算法 2 均终止, 并返回一张无冲突、无死锁且完工时间有限的排程。修复算子永不被调用。

证明. 由构造, 序列是各前后约束链的一个拓扑序, 因为工件的第 x 次出现代表其第 x 道工序; 因此 ready_j 在被读取时总是已知的。缓冲区充裕 (A4) 意味着工序不会因无处存放而被阻塞, 故一旦物料到达, (10) 总给出有限的开工时刻。每次 ROUTE 调用按算法 1 之后的论证在有限时间内返回, 且它只占用预约表报告为空闲的时窗, 所以 (8) 由构造成立。扫描共执行 $N + n\delta_{\text{ret}}$ 次迭代, 每次在有限时刻上安排有限多个事件。□

4.4 预约表感知的车辆派遣

在整个框架中, 选车是唯一一处可以用估计代替事实的地方, 因此我们把它当作一个可闭合的环节, 而不是一个固定的设计。开环规则对每辆车打分

$$\text{score}(k) = \max(\text{avail}_k + t^*(\text{loc}_k, \text{取货点}), \text{ready}_j) + t^*(\text{取货点}, \text{目的地}), \quad (12)$$

它从不查询预约表, 因此可能选中一辆“在地图上很近、实际却被交通堵死”的车。这一规则并非我们为了衬托而虚构的弱基线: 它与文献 [4] 所用的空闲优先规则同式。

作为默认配置的闭环变体改为试探: 对每个候选车辆, 先带提交地规划空载段, 再 不提交地规划满载段, 记下真实送达时刻, 然后把预约表整体回滚。空载段必须真实落表, 否则满载段看不到它占用的时窗, 两段可能被规划到同一走廊的同一时段, 从而给出一个在真实系统中无法实现的乐观值。选定的车辆是实测送达时刻最早者, 而非估计值最优者。

这一机制之所以奏效, 有一个可以说清楚的理由, 它也解释了为什么同样的思路用在选臂上收效甚微。AGV 之间是同构的 (假设 A5), 车与车唯一的差别就在于当前站位与路是否通得过; 因此选车是一个纯粹的拥堵决策, 查询预约表所得到的信息可以全额兑现为目标函数的改善。而选臂除了走廊差异之外还掺入了加工时长, 后者往往压过前者, 信息因而被稀释。

代价是显著的: 试探把每个运输任务的路由调用数从 2 抬到 $2N_A$, 实测单次评价成本约为规则派车的 12.2 倍。若不加处理, 这笔开销在同挂钟预算下会把机制带来的收益恰好吃光——这正是第 4.5 小节存在的理由, 也是第 5 节只在同挂钟预算下比较两者的原因。

4.4.1 层间接口的三种选择. 两层既已固定, 设计问题就变成下层应当回报什么。共有三种答案, 本文实现了全部三种, 并在同挂钟预算下逐一度量。上面的预约表试探属于第一种, 也是唯一一种经得起该检验的; 另外两种的构造在此一并给出, 度量结果见第 5 节。

只回报时间. 下层返回实现的时长, 这正是评价通路已经在做的事, 也是预约表试探所查询的东西。它使目标函数正确, 但不为搜索指方向: 上层只知道某个解慢, 不知道是哪个决策使它慢。在排产之后只路由一次的方法甚至达不到这一点, 因为它们只在决策固定之后知道一次真实时长。值得平实地指出这一选项本身已经达成了什么, 因为本文实测的收益主要归于它: 搜索所比较的每一个候选解, 都是按一个已包含其自身交通所引起延误的时长来排序的。

回报价格. 与拉格朗日松弛及列生成中的协调机制相类比, 自然的下一步是给被争用的资源定价: 为每个“走廊-时段”槽位附加一个影子价格 $\pi(e, b)$, 让路由器在 Pareto 前沿上最小化到达时刻 $+\theta \cdot$ 价格, 并让改派为它所需的通行权付费。我们完整实现了这一方案——既包括一个扰动单个槽位容量并重解的定义式估计器, 也包括一个每代刷新的廉价代理——而在同等算力预算下它系统性地有害。

这一机制值得写出来, 因为它的失效原因可以推广。由争用造成的延误已经被内部化了: 让行的车辆等待, 该等待进入它自己的到达时刻, 再经 (10) 传播进完工时间, 并被适应度计入。对占用者就同一次占用再收一次价格, 等于把它数了两遍。于是车辆为了规避一笔已经支付过的代价而绕行或延迟出发, 以承担一个一阶且确定的自身延误, 去避免一个二阶且不确定的外部性。真正残余的外部性只落在尚未被路由的任务上——扫描是有序的, 早先的预约只可能妨碍其后的任务——而这部分同样已经体现在完整解码后的完工时间里。因此定价只增加扰动, 不增加信息。这个推断是架构层面而非参数层面的: 在任何以完整无冲突解码作为适应度的框架中, 给下层目标再加一个拥堵惩罚项, 很可能是有害的。

技术上还有第二重代价, 它由引理 1 直接给出: 一旦引入价格作为第二判据, 到达时刻不再是全序占优判据, 单标签 Dijkstra 必须换成多标签 Pareto 搜索。无论最终是否真的收取价格, 这套机器都要付钱。第 5 节的度量表明, 击败定价的主要正是这笔开销, 而非上述双重计价的扰动本身。

回报冲突凭证. 第三种选择是传递结构。路由器在发生让行时知道走廊、时段以及占用者任务的身份; 解码器保留让行事件而不只是它们的总量, 因此该信息可以存活到评价结束。一份凭证就是三元组 (走廊, 时段, 对手任务), 它被用来决定算子该触碰哪些工序。关键在于它不进入任何目标函数: 下层依旧最小化到达时刻, 上层依旧最小化完工时间, 改变的只是邻域的构造方式。既然没有任何量被加入, 也就没有任何量被数两遍——这避免了击沉定价的那个缺陷。第 4.6 小节把它落实为算子。它最终同样没有产出净收益: 它构造出的邻域瞄得很准, 但成功率过低, 不足以抵偿它消耗的解码次数。我们完整给出它, 是因为“构造上站得住却依然不划算”的机制比只报告成功者更有信息量——会自己设计出它的读者可以直接看到它的代价。

4.5 降本: 可采纳下界剪枝与胜者路径复用

上一小节的机制在同挂钟预算下面临一个具体的困境。按相同代数比较, 预约表试探比规则派车约好 3%; 但它单次评价贵 12.2 倍, 于是在相同挂钟时间下这 3% 恰好被它自己消耗的算力吃光, 净效果几乎为零。换言之, 在这个问题上降本就是增效: 只要能在不改变任何选择结果的前提下压低成本倍数, 机制的收益就能显现出来。本小节给出两项这样的优化, 它们都可证不改变输出。

可采纳下界剪枝。关键观察是: 无冲突路由只会因让行而更晚, 绝不可能快过忽略争用的理想最短路。因此开环规则所用的估计 (12) 恰好是实测送达时刻的一个可采纳下界:

$$lb(k) = \max(avail_k + t^*(loc_k, \text{取货点}), ready_j) + t^*(\text{取货点}, \text{目的地}) \leq est(k), \quad (13)$$

其中 $est(k)$ 为对车辆 k 做完整两段试探所得的实测送达时刻。于是, 若某辆车的下界已经不优于当前最优的实测值, 它的实测值必然也不优, 便无需为它调用路由。被剪掉的车辆一次路由调用也不产生。

胜者路径复用。试探结束后预约表被整体回滚, 连胜者的两段路径也一并作废, 随后解码器又把它们重算一遍, 白白浪费两次路由调用。而 $ROUTE(\cdot, commit = \text{真})$ 所做的不过是把各段依次写入预约表, 因此把胜者在试探阶段已算出的路径缓存下来、直接提交即可, 无需重算。

命题 3 (两项优化保持输出不变)。在按车辆编号升序扫描并保留首个严格更优者的实现下, 加入 (13) 的剪枝与胜者路径复用之后, 所选车辆、所提交的预约以及最终时间表, 与全量试探逐位相同。

证明。先看剪枝。设扫描到车辆 k 时当前最优实测值为 est^* 。若 $lb(k) \geq est^*$, 则由 (13) 有 $est(k) \geq lb(k) \geq est^*$, 故 k 不满足“严格更优”的替换条件, 在全量试探中同样不会成为胜者; 跳过它不改变胜者。又因每次试探都以检查点开始、以回滚结束, 被跳过的试探对预约表不留任何痕迹, 故也不改变后续任何一次查询的结果。再看复用: 胜者的两段路径是在同一张预约表状态下算出的, 而回滚把表恢复到了试探开始前的状态, 与解码器随后提交时所面对的状态一致; 由于 $ROUTE(\cdot, commit = \text{真})$ 仅把给定各段写入预约表, 提交缓存路径与重算后提交产生的占用区间完全相同。 $\square$

两项优化合计带来 2.33 倍加速, 路由调用减少 47.4%, 把单次评价的成本倍数从 12.2 压到 5.23。这不是工程细节, 而是使第 5 节的正面结论得以成立的前提: 在降本之前, 同一机制在同挂钟下的净效果无法与噪声区分。

剪枝的效力本身依赖于车队规模, 这一点须如实说明。下界越紧、越早出现一个好的现任值, 被剪掉的车辆就越多; 而车队越大, 候选越多, 试探的总开销随 N_A 线性增长。第 5 节给出实测的代价律, 并指出在车队足够大且争用足够强时剪枝会失效。

4.6 冲突制导局部搜索

本小节把第 4.4.1 小节的第三种接口落实为算子。每一代中, 种群里最好的 10% 接受局部搜索。需要提醒读者的是, 这一机制在同挂钟预算下未能转为净收益 (第 5 节); 此处完整给出其构造, 是为了让“为何最终选定预约表试探”这一取舍有据可查。

4.6.1 关键链的归因。从取到完工时间的那个事件向后回溯可以得到一条链, 但仅由工序构成的链无法说明一次开工为何迟到。当 $bind = arrive$ 时, 该工序是在等物料, 而成因可能是上游工序、可能是无车可用、也可能是交通拥堵——三种情形对应三种完全不同的补救手段。因此我们把运输段也纳入回溯, 并给每一环打上标注 (表 3)。满载段上的让行一律记为走廊环, 因为它直接延长了到达时刻; 空载段上则按让行时间占该段总耗时的比例、以 0.5 为阈值, 在“车辆”与“走廊”之间划分。

从标注后的链上导出三类查询: 链上的真实工序, 改派算子从其中抽取候选; 链上的“走廊-时段”槽位; 以及对给定走廊与时段, 曾占用它并因而造成让行的那些任务——这才是凭证本身。

4.6.2 两族算子。拥堵有两种补救方向, 框架为每一种提供一个算子。

改派改变“地点”。对关键链上至多 $L = 5$ 道真实工序, 为每个备选机械臂 $m' \in \Omega_{ji} \setminus \{m\}$ 打分

$$score(m') = t^*(pos_{j,i-1}, loc(m')) + t_{jim'}^P, \quad (14)$$

表 3. 关键链各环节的标注及其对应的补救算子。只有最后一类携带具体的走廊与时段,也只有它能指名对手任务。

标注	判定条件	补救手段
operation	一道真实工序正占用其机械臂	—
machine	bind = machine, 被该臂上前一道工序阻塞	重排序 (通用变异)
upstream	车辆先到, 物料尚未就绪	递归至 $O_{i,j-1}$
vehicle	物料已就绪、车辆迟到, 且延误中让行占比很小	更换车辆
corridor	在指定走廊、指定时段上发生让行	改派或错峰

算法 3 CONTENTIONGUIDEDSEARCH: 冲突制导的局部搜索

Require: 染色体 c 及其解码结果 r

```

1: for round = 1 到 3 do
2:    $\mathcal{N} \leftarrow \text{REASSIGN}(c, r) \cup \text{STAGGER}(c, r)$  ▷ 至多  $L + 2$  个邻居, 均由凭证指名
3:   improved  $\leftarrow$  假
4:   for all  $c' \in \mathcal{N}$  do
5:      $r' \leftarrow \text{DECODE}(c')$ 
6:     if  $r'.C_{\max} < r.C_{\max}$  then
7:        $c \leftarrow c'; r \leftarrow r'; \text{improved} \leftarrow$  真; 跳出
8:     end if
9:   end for
10:  if 非 improved then
11:    跳出
12:  end if
13: end for
14: return  $c, r$ 

```

即一个邻近项与一个工时项之和, 两者同为时间单位; 得分最优的机械臂产生一个仅有单基因差异的邻居。这两项恰好就是本文所讨论的那个权衡: 更近的机械臂可能更慢, 而 (14) 正是使搜索能够在“通往快速机械臂的路径被争用”时, 用加工时间去买邻近性的那个式子。

该打分的一个早期版本还加上了 λ 乘以候选机械臂处累计的让行时间。那一项是一个横跨整个计划期与整个车队的总量, 它随算例规模增长, 而前两项始终停留在单道工序的尺度上; 在一个 240 道工序的算例上它会支配整个打分, 算子随之退化为“永远选最不拥堵的机械臂”, 从而毁掉它本要表达的那个权衡。我们报告这一点, 是因为 λ 在不同算例之间不可比, 对它做任何敏感性分析都将是无意义的。

错峰改变”时间”。取链上最长的 corridor 环, 向凭证查询在该时段占用过该走廊的任务。由此得到两个有指向的邻居: 把被阻塞的工序在序列中提前, 使其移动被投放到一个更空闲的时窗; 或把单个对手工序推后, 使两者不再重合。每一轮只挪动一个对手, 以保持邻域小而有针对性。在序列中移动一次出现, 是与另一个工件的基因交换位置, 因此工件内部次序自动得以保持, 结果仍是一个合法的可重排列; 由命题 2, 它因而仍然可解码。

于是邻域最多包含 $L + 2 = 7$ 个候选, 且每一个都是由下层指名的。这正是它与通用变异算子的具体差别——后者会在同一空间中均匀随机地抽样。

4.6.3 接受准则. 第一个改进完工时间的邻居立即取代现任; 随后从新的现任重新提取关键链, 该轮重复, 最多三轮。某一轮若未产生任何改进则结束搜索 (算法 3)。重新提取是必要的: 一次成功的移动之后, 支配完工时间的那个成因通常会转移, 因此若只在开始时算一次链, 后续各轮就会去追一个已经不存在的瓶颈。

4.7 外层遗传算法

基于种群的这一层刻意保持常规, 以使任何实测增益都可归因于上述机制而非算子调参。初始种群随机生成——机械臂自 Ω_{ji} 均匀抽取, 序列随机打乱——但另植入两个个体, 一个把每道工序都指派给其最快的机械臂, 一

个按负载均衡贪心指派。选择采用二元锦标赛并保留五个精英。交叉对序列段使用保持前后约束的顺序交叉,对指派段使用均匀交叉;变异或交换序列中两个位置,或把一道工序移到另一台可选机械臂。适应度即解码所得的完工时间。这些选择都是标准做法,不作为本文贡献。

4.8 复杂度与代价分析

一次适应度评价扫过 $N + n\delta_{\text{ret}}$ 个事件;在闭环派遣下每次移动花费至多 $2N_A$ 次路由调用,故每个个体为 $O((N + n)N_A|E|W \log|V|)$;开环规则以一个估计值为代价去掉了因子 N_A 。第 4.5 小节的下界剪枝不改变这一最坏情形界,但按命题 3 在不改变输出的前提下把实测的调用数降低了 47.4%,即把因子 N_A 换成了一个显著更小的有效值。局部搜索则为它触碰的每个个体再增加至多 $3(L + 2)$ 次解码。

其后果是,本框架内不同配置的成本相差达两个数量级——实测每次评价:两阶段配置 0.39 ms,完整闭环 15.4 ms,加上定价则为 77.9 ms。因此按相同代数比较是没有意义的,第 5 节改为对齐挂钟时间。

这不是记账层面的细节,而是实验章围绕成本而非围绕机制链来组织的原因。在固定预算下,单次评价的成本与评价的次数是同一份资源的两种花法。一个机制只有在“它每次评价所增加的收益”超过“它挤占掉的那些评价本可贡献的收益”时才配得上它的位置,而这是一个任何设计层面的推理都无法预先判定的定量检验。本文的三个候选接口中,只有预约表试探通过了这一检验,且它只有在第 4.5 小节的降本之后才通过。

最后是一个由框架结构本身引出的声明。交换乘子或价格的双层方案有时可以被证明收敛到不动点;本框架不能,我们也不作此主张。下层是优先级式的而非联合最优的,上层是在非凸组合空间上的随机搜索,而层间接口传递的是证据,不是一个可以充当 Lyapunov 函数的量。命题 2、引理 1 与命题 3 所保证的东西更窄,但对一个启发式方法而言更有用:搜索考察的每一个候选解都是可行的,其评价相对第 3 节的冲突模型是精确的,降本没有改变任何一个选择,并且整个过程是可复现的。解的质量是一个经验问题,由下一节在一个为保持诚实而设计的协议下回答。

5 实验与结果分析

本章回答四个问题。把拥堵搬进搜索目标(闭环)本身值多少?预约表感知的派遣再值多少,它能否代替闭环、二者是正交互还是近加性?这些收益在多大的算例范围内成立、适用上界在哪?以及,那些我们实现了却未能转为净收益的更复杂接口,失效在哪一环?

5.1 实验设置

5.1.1 受控算例族。公开的 FJSP_PT 基准把行驶时间给成一张“位置对-时长”矩阵,而这恰是本文要去掉的那条假设;它们不含通道网络,车辆争用在其上无法定义。因此我们生成一族受控算例,其中网络是显式的,且第 3.7 小节判定为关键的那些量可以逐个单独变化。

全部算例共用同一核心:12 个工件、每工件 3 道工序共 36 道真实工序、8 台机械臂、异质度 $H = 0.3$,且运输与加工的时间尺度比 $\bar{T}_t/\bar{T}_p$ 标定到 4.0——即运输是矛盾的主要方面,这正是本文问题设定的用意所在。在此核心之上变化三族因子(表 4)。

A 族:布局(争用结构)。五档 funnel、high、mid、low、scatter,通过改变网络容量而不改变行驶距离来调节争用:进入车间与穿越车间的并行通道数逐档增加,每条通道是总时长相等的两跳路径,故增删一条只改变“多少辆车能并行通过”,不改变“任何东西要走多远”。节点与走廊数由 13/12 递增至 16/24。其中 funnel 与 high 只差站台处的一条并行通道,即最小割减半,是本章最干净的受控对照;low 与 scatter 网络规模相同,只差机械臂的空间摆放。

B 族:车臂比 N_A/N_M 。在 funnel 布局上取 $N_A \in \{4, 8, 16\}$,即车臂比 0.5、1.0、2.0。这一族是为第 5.7 小节的代价律准备的:派遣试探的开销随车队规模线性增长,而剪枝的效力随之下降,两者的交叉点决定机制的适用上界。

表 4. 受控算例族。全部算例格共用 12 工件、36 道真实工序、8 台机械臂、 $H = 0.3$ 、 $\bar{T}_l/\bar{T}_p \approx 4.0$; 三族分别只变化布局、车臂比与柔性。争用强度为初始随机种群下让行时间占行驶总时间的比例, 由算例自身结构决定而非人为设定。

算例	变化的因子	节点	走廊	N_A	F	争用强度	复合下界
A funnel	布局	13	12	12	0.61	31.9%	24.0
A high	布局	14	14	12	0.61	30.9%	17.0
A low	布局	16	24	12	0.61	11.8%	36.0
A mid	布局	15	16	12	0.61	10.8%	17.0
A scatter	布局	16	24	12	0.61	6.6%	36.0
B NA/NM 0.5	车臂比	13	12	4	0.61	15.8%	24.0
B NA/NM 1.0	车臂比	13	12	8	0.61	30.2%	24.0
B NA/NM 2.0	车臂比	13	12	16	0.61	32.4%	24.0
C F=0.3	柔性	13	12	12	0.30	27.7%	24.0
C F=1.0	柔性	13	12	12	1.00	33.5%	24.0

C 族: 柔性 F 。在同一布局上取 $F \in \{0.3, 1.0\}$, 即每道工序的可选机械臂比例。柔性决定改派算子有多少可动的余地, 也决定拥堵信息能在多大程度上转化为指派上的调整。

三族共 10 个算例格, A funnel 作为三族共用的基点只跑一次。

5.1.2 比较协议. 这个设定下有三个陷阱会使朴素的比较近乎无意义。我们把协议写得较细, 因为其中两个是我们先踩进去、之后才改过来的。

统一执行器: 报可实现的完工时间。这是公平性的支点, 也是与文献比较时最容易出错的一处。开环方法在忽略争用的前提下排产, 它自己报出的完工时间在真实系统里根本不可行。因此本章的所有配置, 无论计划从哪里来, 都必须放进同一个无冲突路由器里执行、过同一个独立校验器, 报告的一律是执行后可实现的完工时间。这一步不是形式主义: 开环计划的乐观幅度恰等于该算例的争用强度, 在漏斗算例上可达 33.5%。若允许各方各报自己口径下的数, 比较立即失去意义, 而被比较的一方会凭一个不可实现的数字胜出。

同挂钟, 而非同代数。各配置评价的不是同一个对象。开环配置的一次评价是查常数矩阵 t^* 的解码; 闭环配置要对每个候选做完整的无冲突路由; 定价变体还要额外跑多标签搜索。实测单次评价成本跨越两个数量级。因此按相同代数比较等于给昂贵的配置多送几十倍算力, 一个看似机制增益可能只是预算更大。我们改为对齐挂钟时间: 各配置在同一算例上共享同一预算, 早停阈值放宽以确保预算被真正用尽, 每次运行都报告它是耗尽预算还是提前收敛。

对齐时间同样不是中性的——它反过来给便宜的配置多送几十倍搜索次数。因此两种口径都不能单独呈现。我们为每个配置同时报告单次评价成本与实际完成的评价次数, 使搜索努力的差异可见而非被藏起来, 并在第 5.7 小节把这一“成本与次数的兑换率”本身作为对象来测。

足够的种子与配对检验。本问题中单种子读数是没价值的: 在同一算例上, 同一配置的三个种子之间我们观察到过 11 个完工时间单位的极差, 超过我们所度量的任何机制效应。因此每格跑 10 个种子, 每个比较都按种子配对, 并用双侧 Wilcoxon 符号秩检验。配对按显式键(算例, 种子)形成, 而非按列表位置——这个区别在某格数据不全时是决定性的。

正确性。每个解都由一个独立于解码器的校验器重新检查, 该校验器从时间表出发重新推导走廊占用, 并与算例的复合下界比对。未过校验或低于下界的运行单独报告, 绝不并入任何格的均值。每次完成的运行都追加写入账本并落盘, 故中断的批次可以续跑而不重算, 报表也从账本而非内存重建。

表 5. 四级阶梯作为 2×2 析因。行为“拥堵是否进入搜索目标”,列为“派车是否查预约表”。四格中只有沿同一行或同一列的比较是单因子的,对角比较跨两个因子、不可用于归因。

	规则派车	预约表感知派车
开环规划 (排产冻结)	B0 [4]	B0+ [7]
闭环规划 (拥堵入目标)	B1	B2(本文)

5.1.3 实现与运行环境. 实验在 Intel Xeon W-1270 @ 3.40 GHz 上单线程运行; 实现为纯 Python, 不依赖任何求解器。因此绝对耗时不能与编译型实现相比, 这一点对协议是有影响的: 更快的实现会让每一个配置在同一预算内做更多次评价。但它不会改变各比较所依赖的“成本与评价次数之间的兑换率”, 因为该比率是在共享同一实现的配置之间测得的。

5.2 四级基线阶梯

一体化调度类工作的实验最常见的失误, 是把两个不同的问题混成一个数字来报: 与文献相比好多少, 以及我们自己的哪个部件贡献了多少。前者要求与已发表方法的结构对齐, 后者要求只动一个部件。只报消融会低估问题的分量, 只报文献比较则会把无冲突路由本身的价值冒领成本文的创新。我们因此把它拆成一道四级阶梯, 每一级都对应一个可指名的结构。

B0: 开环规划 + 真实执行。 上层在理想最短路搜索 (δ 冲突不计), 搜完把计划放进真实路由器执行。这是 [4] 的忠实模型: 其调度层产出忽略运输争用的甘特图, 下游用优先级路由解冲突, 再把到达时刻平移回时间轴; 其派车用空闲优先规则 $k^* = \arg \min \text{Cost}(\text{path}_k, L_{\text{start}})$, 与本框架的规则派车 (12) 同式。

B0+: 开环规划 + 预约表感知派车。 排产同样冻结, 只把车辆指派改为查预约表决定。这是 [7] 两阶段结构的对应物: 其第一阶段的混合遗传算法在常数运输时间矩阵上排产, 第二阶段才在冻结的排产上优化车辆指派并处理冲突, 无回流。此处以试探派车作其第二阶段的贪心近似——试探未必弱于对方的遗传算法, 故这一级是对基线宽容的口径。

B1: 闭环规划 + 规则派车。 搜索时即计冲突, 派车仍用理想矩阵估计。与 B2 信息完全对等。

B2: 闭环规划 + 预约表感知派车。 本文方法。

这四级不是一条链, 而是一个 2×2 析因: 因子一为“拥堵是否进入搜索目标”(开环/闭环), 因子二为“派车是否查预约表”(规则/试探)。按析因而非按链来组织是必要的, 因为链上相邻两行只有在此前每个因子都被固定时才只差一个因子; 一旦两行差了二个因子, 它们的差值就不可解释。具体地, $B0+ \rightarrow B1$ 同时跨越两个因子, 不能用度量闭环的价值——我们把它列出仅为对账。干净的对比如是固定一个因子只动另一个, 即表 5 的四格之间的四个配对。

B0 的执行口径需要交代, 因为它直接影响基线的强弱。开环搜出的计划本身含有一套派车决策, 执行时有两种复现方式: 复现计划用原计划逐字指定的车辆, 即“计划怎么定就怎么执行”; 现场改派在执行时按规则重新派车。实测前者更强 (漏斗算例上 63.00 对 78.00), 且它才是 [4] 的忠实模型——该文的任务分配层在甘特图上先定好机器人, 执行滑坡时只平移时间轴、不改派。因此我们取较强的那一个作为主报口径, 另一个一并给出以示我们没有挑对自己有利的那个。对基线取强者, 是举证责任应当由我们承担的部分。

5.3 主结果

表 6 给出四级阶梯在 10 个算例格 $\times$ 10 个种子、共 100 个配对下的结果, 每格同挂钟预算, 全部运行均通过独立校验。以下三段分别读出析因的两个主效应与端到端的合计。

表 6. 四级阶梯在同挂钟预算下的完工时间, 以及 2×2 析因的四个单因子配对比较。负值表示完工时间下降, 即更好。 $n = 100$ 个按 (算例, 种子) 配对的比较, 双侧 Wilcoxon 符号秩检验。

算例	争用强度	B0	B0 ⁺	B1	B2
A funnel	31.9%	58.10	53.50	44.70	42.40
A high	30.9%	56.00	51.40	39.90	37.70
A low	11.8%	64.50	64.60	57.80	57.10
A mid	10.8%	46.40	42.70	39.40	37.40
A scatter	6.6%	62.30	62.30	57.20	57.10
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20
C F=0.3	27.7%	74.30	69.50	51.60	47.30
C F=1.0	33.5%	52.10	47.50	39.30	36.30
全体均值	—	61.20	58.27	48.86	47.44

单因子配对对比	$\Delta C_{\max}$	胜/负/平	p
闭环的价值 (规则派车下)	-18.86%***	96/2/2	$< 10^{-4}$
闭环的价值 (试探派车下)	-17.75%***	93/5/2	$< 10^{-4}$
试探派车的价值 (开环下)	-4.59%***	68/19/13	$< 10^{-4}$
试探派车的价值 (闭环内)	-2.45%**	60/34/6	0.0054
端到端: 文献结构 $\rightarrow$ 本文	-21.35%***	94/5/1	$< 10^{-4}$

闭环本身的价值。· 固定派车方式、只改变“拥堵是否进入搜索目标”, 闭环在两种派车方式下都显著更优: 规则派车下 $B0 \rightarrow B1$ 为 -18.86% ($p = 0.0000$), 试探派车下 $B0+ \rightarrow B2$ 为 -17.75% ($p = 0.0000$)。这个增益不能归因于搜索努力, 因为比较方向恰恰相反: 开环配置的一次评价只需查常数矩阵, 在同一预算内能完成数倍于闭环的评价次数。它搜得更狠却仍然输, 原因是它在优化一个错误的目标, 只有当决策已被冻结之后才发现真实的运输代价。图 2 在单个算例上把这一差距画了出来。

值得指出这一级差距的归属。把拥堵搬进搜索目标是本文的主张, 同时也是 [4] 在其结论中自陈的未来工作 (“extending this mechanism into a fully iterative loop”), 因此可以正当主张; 但它必须与下一段的机制贡献分开报, 不得合并成一个数字。

预约表感知派遣的价值。· 固定闭环与否、只改变派车决策, 得到本文机制自身的贡献。在闭环内部, $B1 \rightarrow B2$ 为 -2.45% ($p = 0.0054$)。这一级的两档信息完全对等——同样的排产搜索、同样的预算, 唯一差别是选车时是否查预约表——因此它是本文承重机制的干净度量。该均值被车队偏大的两格拖低: 那两格上试探反而抬高完工时间 (+4.8%, +4.7%), 故 -2.45% 是含适用上界在内的平均, 不是处处成立的点估计。

在开环之下, 同一个机制值 -4.59% ($p = 0.0000$), 同样显著。开环试探并不“几乎不值钱”; 它就是 [7] 两阶段结构相对 [4] 忠实开环的那一段进步。但它的量级 (4.59%) 远小于闭环主效应 (18.86%), 这才是下一小节要钉死的判断。

端到端。· 从文献结构到本文方法, $B0 \rightarrow B2$ 合计 -21.35% ($p = 0.0000$, 94 胜 5 负)。这是一个可以正当主张的端到端数字, 前提是它被拆开报告过——上面两段就是那个拆解。

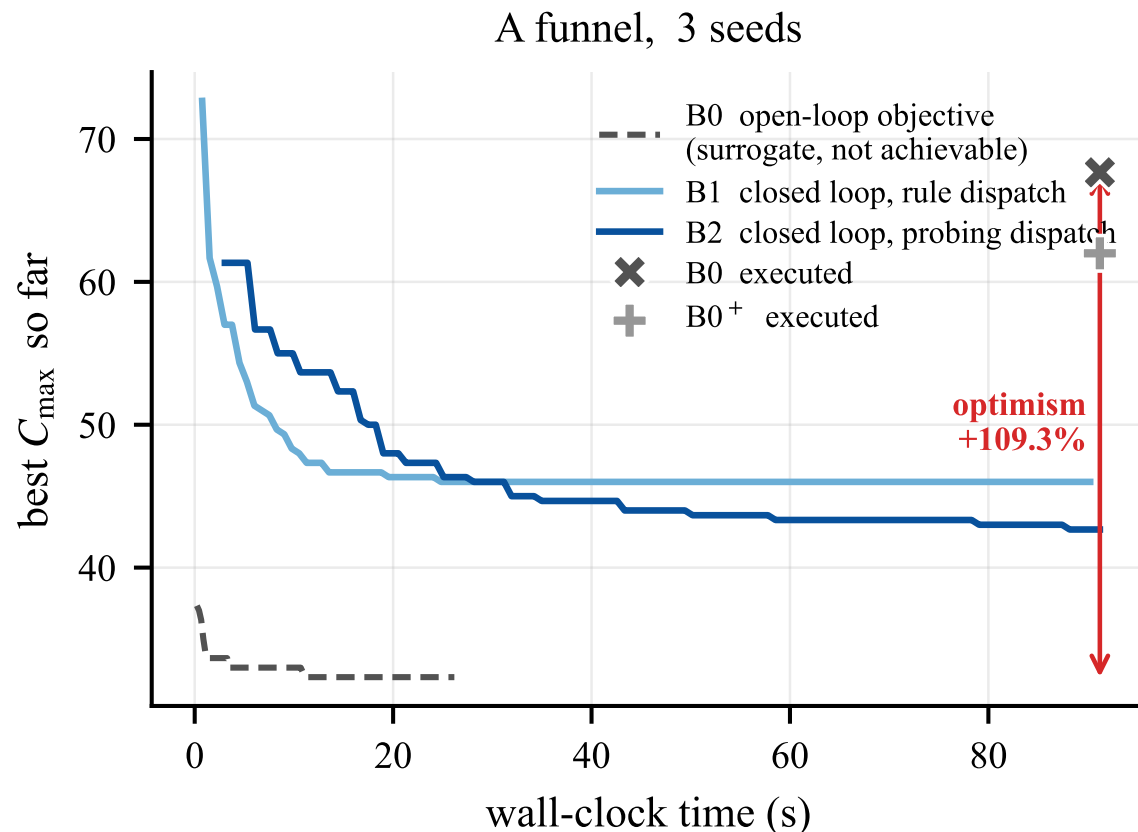


Fig. 2. 单个 funnel 算例上的收敛曲线, 横轴为挂钟时间, 曲线为多种子的逐时刻均值。实线 (B1、B2) 自始至终画的都是可实现的完工时间; 虚线是开环搜索自己的代理目标, 即它实际在优化的东西, 而叉号与十字标出同一份计划经无冲突路由执行后落在哪里。虚线终点与叉号之间的落差即该算例的乐观幅度, 也是“在环内评价”这一步的全部理由: 开环并非收敛得不好, 而是收敛到了一个不存在的位置。

5.4 主效应、次效应与不可替代性

上一小节留下的不是“开环试探不显著”, 而是一个量级问题: 开环试探显著但小 (4.59%), 闭环显著且大 (18.86%), 闭环内试探的平均被大车队两格拖成 2.45%。本小节用析因把“可否互相替代”与“是否正交互”分开回答。

交互的定量判据。若闭环与派遣两个因子彼此独立, 端到端增益应恰为两者单独增益的复合, 即 22.6%。实测为 21.35%, 略低于独立预测——无正交互, 呈近加性或略亚加性。亚加性与第 5.5 小节的适用上界同向: 大车队两格上闭环内试探为负, 把组合增益往下拽。

不可替代, 而非必须捆绑。真正站得住的主张不是“两者必须一起用才有效”, 而是“开环试探不能代替闭环”。把拥堵感知的派车装在冻结排产上, 最多拿到约 4.59%; 把拥堵搬进搜索目标, 即使仍用规则派车, 也能拿到约 18.86%。熟悉 [7] 的审稿人若问“你们的派车机制装在两阶段框架里是不是也一样管用”, 答案是: 管用, 但只管用开环试探那一档; 要拿到本文这一级的改善, 拥堵必须在搜索时进入目标。B2 仍是端到端最优, 因为它把主效应与 (在适用区间内的) 次效应叠在一起, 而不是因为存在一个把两者焊死的正交互项。

表 7. 逐算例格的四级阶梯结果与两个主效应, 按三个因子族分组 ($n = 10$ 个按种子配对的比较每格)。争用强度由算例结构决定, 一并列出以便判断收益是否随其变化。负值表示完工时间下降。

算例	争用强度	B0	B0 ⁺	B1	B2	B0→B1	B1→B2
A funnel	31.9%	58.10	53.50	44.70	42.40	-23.1%	-5.1%
A high	30.9%	56.00	51.40	39.90	37.70	-28.8%	-5.5%
A low	11.8%	64.50	64.60	57.80	57.10	-10.4%	-1.2%
A mid	10.8%	46.40	42.70	39.40	37.40	-15.1%	-5.1%
A scatter	6.6%	62.30	62.30	57.20	57.10	-8.2%	-0.2%
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10	-2.8%	-5.7%
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80	-16.2%	+4.8%
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20	-40.2%	+4.7%
C F=0.3	27.7%	74.30	69.50	51.60	47.30	-30.6%	-8.3%
C F=1.0	33.5%	52.10	47.50	39.30	36.30	-24.6%	-7.6%

对文献的含义。因此“先排产、后解冲突”路线的问题, 不在于第二阶段的派车做得不够好——第二阶段在我们的口径下确实有 4.59% 的显著收益——而在于它把主效应留在了桌面上。第二阶段能挽回的东西有一个由第一阶段无知所框定的上界; 要突破这个上界, 必须把无冲突执行放进适应度。

5.5 稳健性与适用区间

主结果是在 10 个算例格上汇总的。本小节把它按三个因子族拆开, 目的不是再多报几个数, 而是划出机制的适用区间: 一个只在特定结构上成立的收益, 若不说明边界, 读者无法判断它是否适用于自己的场景。三族分别检验三个可以事先说清的预期。

布局 (A 族) 检验收益是否随争用强度增长。这是最直接的预期: 闭环与预约表试探处理的都是争用, 若争用本身很弱, 两者都无从发挥。funnel 与 high 只差站台处一条并行通道, 即最小割减半, 是最干净的受控对照。

车臂比 (B 族) 检验上界。派遣试探的开销随车队规模线性增长, 而第 4.5 小节的剪枝效力随车队增大而下降 (等价性自检中最大车队一档的调用削减率最低)。因此存在一个车队规模, 超过它之后机制的收益会被自身开销吞掉。这个上界必须报出来。

柔性 (C 族) 检验机制是否依赖指派上的可动余地。柔性越高, 同一道工序的可选机械臂越多, 拥堵信息越有机会转化为指派调整。

适用上界: 大车队两格上试探反而有害。表 7 把 B 族的预言钉成了数字。车队最紧的一格 ($N_A/N_M = 0.5$) 上, 闭环仍有益 ($B0 \rightarrow B1$ 为 -2.8%), 而试探的收益最大 ($B1 \rightarrow B2$ 达 -5.7%)——车辆稀缺时, 选对那一辆几乎决定完工时间, 查预约表的信息全额兑现。反过来, 在 $N_A/N_M = 1.0$ 与 2.0 两格上, 闭环内试探抬高完工时间 (+4.8%, +4.7%); 单次评价因多车试探而变贵, 同挂钟内搜得更少, 收益被开销吃光并反超。这正是第 4.5 小节代价律的适用上界形态, 也解释了为何闭环内试探的平均增益只有 2.45%——均值被这两格往上拽。机制有效的完整主张必须带上这句话: 在车队不过大时有效; 超过上界则应退回规则派车。

表 7 与图 3 给出逐格结果。三族的读法各不相同, 须分开说。

布局族。闭环增益随争用强度大致上升: 最不拥堵的 scatter (争用 6.6%) 上 $B0 \rightarrow B1$ 为 -8.2%, 中等的 mid/low 约 -15%/-10%, 最拥堵的 funnel/high 达 -23%/-29%。试探增益同向但幅度更小, 并在低争用两格上接近零 (-0.2%、-1.2%)。机制处理的确实是争用, 而不是某种与争用无关的搜索副产物。受控对照 funnel 对 high (只差站台一条通道) 上两者都大, 与“最小割减半抬高可规避争用”的构造一致。

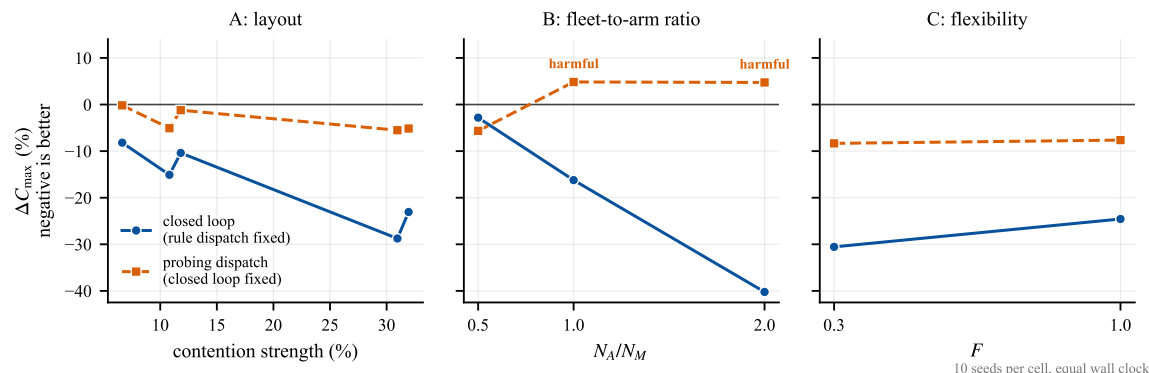


Fig. 3. 两个主效应在三个因子族上的分布。左: 布局族, 横轴为争用强度; 中: 车臂比族, 横轴为 N_A/N_M ; 右: 柔性族。闭环增益在各族上稳健且大体随争用上升; 闭环内试探派车在车队最紧时最赚, 在 $N_A/N_M \geq 1$ 两格上翻为正 (图中标注 harmful), 该转折即适用上界。

车臂比族。这是适用上界被钉死的一处。 $N_A/N_M = 0.5$ 时闭环内试探最赚 (-5.7%); 到 1.0 与 2.0 则翻为有害 (+4.8%, +4.7%)。我们事先预期的是“中等车队处最大、两端更小”的驼峰, 实测峰值落在最紧一端、大车队直接变号——比驼峰更干脆, 也更符合代价律: 车少时选对车几乎决定完工时间, 信息全额兑现; 车多时试探次数线性膨胀, 剪枝效力下降 (第 4.5 小节自检中 $N_A/N_M = 2.0$ 的调用削减率六格最低), 同挂钟内搜得更少。闭环本身在大车队上反而更大 (-16%、-40%), 说明上界卡的是试探, 不是闭环。

柔性族。试探增益在 $F = 0.3$ 与 $F = 1.0$ 上接近 (-8.3% 与 -7.6%), 闭环增益同为大幅负值。拥堵信息主要经由派车与时序兑现, 并不强依赖“还有没有另一台臂可改派”。这对机制解释是利好: 它不是柔性车间专属的旁路, 在指派余地受限时仍然成立。

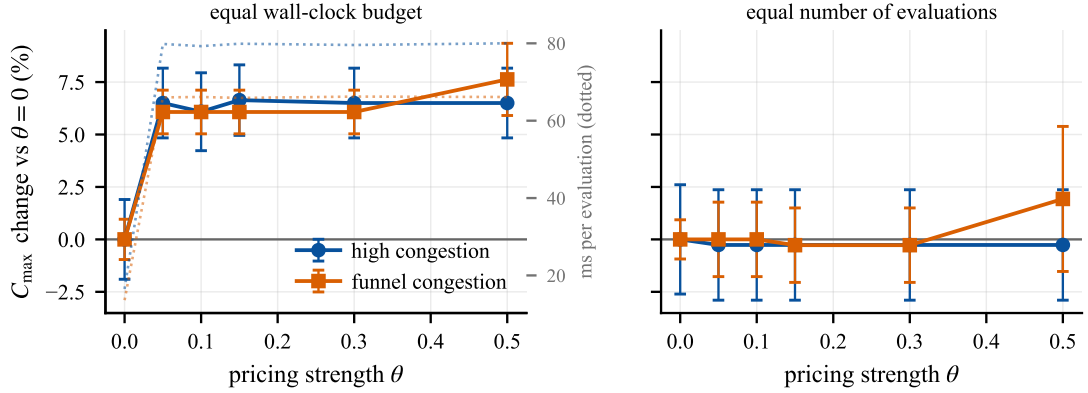
来自另一批算例的独立印证。上述结论建立在一个算例族与一个基线口径之上, 因此我们用一批独立的数据交叉核对。该批次为 16 个算例格 $\times$ 10 种子、同挂钟预算、共 1280 次运行, 算例核心与基线口径都与本章不同——它以两阶段代理运输作为基线, 并在四个争用族与四个异质度水平上交叉。其集成增益 (两阶段 $\rightarrow$ 闭环) 为 5.52%, 16 格中 11 格显著, 区间 0.28% 到 9.28%。这与本章闭环主效应的方向与量级一致, 且是在不同算例、不同基线下得到的, 故可作为稳健性的独立证据。

同一批数据对预约表感知派遣给出的却是 -0.74%、16 格中 0 格显著, 与本章的 2.45% 方向相反。这一表面矛盾有确定的成因, 它同时是下一小节的主题: 该批次运行于第 4.5 小节的降本措施引入之前, 当时同一机制的开销尚未被压下来。

5.6 消融与受控负对照

第 4.4.1 小节列出了层间接口的三种选择, 本文只把第一种 (预约表试探) 放进了默认配置。另外两种都被完整实现并度量, 结果是它们在同挂钟预算下未能转为净收益。本小节报告这些度量。它们在本文中的地位是设计取舍的证据, 而不是一级结论: 一个构造上站得住却仍不划算的机制, 恰好说明为什么最终选定的是那个更简单的。

5.6.1 冲突凭证制导的局部搜索 在第 5.5 小节引用的那批 16 格数据中, 关掉冲突凭证反馈的相对增益为 -1.72% (16 格中仅 1 格显著), 关掉错峰算子为 -0.16% (2 格显著)。两者都不为正, 即在同挂钟预算下, 把算力花在这个邻域上不如把它还给种群循环。第 5.6.3 小节给出机制层面的解释。



positive is worse. 20 runs per point on the left, 10 on the right; two instances, mean $\pm$ s.e.

Fig. 4. 为层间接口定价。正值表示更差。在同挂钟预算下 (左) 惩罚在定价一开启时即出现, 随后在 θ 的十倍范围内保持平坦, 并与右轴的单次评价成本同步。在同评价次数下 (右) 惩罚消失。因此这种损害是“路由搜索变贵”的记账后果, 而非价格误导了路由器的证据。

需要说明的是这两个消融为何可以直接引用那批更早的数据, 而派遣的消融不可以: 凭证与错峰这两个机制在降本前后没有改动, 其开销也不随剪枝变化, 故那批测量对它们依然有效; 而派遣的开销正是降本所针对的对象。

5.6.2 走廊时段定价. 第 4.4.1 小节论证了给走廊占用定价会把适应度已经看到的延误数第二遍。该论证预言定价不能增加信息, 但它本身并不说明实测的惩罚会有多大, 而把这两件事分开恰恰是要紧的。我们在两种预算口径下扫描协调强度 θ , 并在所有 θ 上固定路由器的搜索宽度, 以免定价与“路由器考察多少个进入时刻”相混淆。

实测的形态见图 4: 同挂钟下定价在我们试过的每个 θ 上都使完工时间变差且显著, 且惩罚在 θ 的十倍范围内保持平坦; 同评价次数下同一比较与零无法区分。解释在评价次数里——带价格的路由在同一预算内完成的评价次数只有不带价格时的约五分之一。定价并没有把候选解排错序, 它是把搜索饿死了。

两项补充观察排除了其他读法。其一, θ 在本问题上不是一个有意义的旋钮: 在跨两个数量级的范围内, 带价格的路由由改变的是同样那几条路径、付出同样的总价格、返回同样的完工时间; 累计价格仅占总行驶时间的很小一部分, 这个扰动太小, 不足以推翻到达时刻在标量化目标 到达时刻 $+\theta \cdot$ 价格 中所施加的次序。只有当 θ 大到价格压过到达时刻时, 双重计价的损害才真正显现——而那时完工时间大幅恶化。其二, 开销是结构性的, 而非价格非零所致: 同一次解码在带价格与不带价格下的耗时之比在每个 θ 上都相同, 因为这笔钱是为“每个节点维护一个 Pareto 前沿、每条弧尝试若干进入时刻”付的, 与是否真的收取价格无关。这正是引理 1 所预言的: 引入第二判据即摧毁全序占优, 单标签搜索必须让位给多标签搜索。

我们把这个负结果写得较细, 因为它比一个正结果更可迁移, 也因为两种预算口径对它的幅度有分歧而对符号没有。其中与实现无关的那部分论证是: 无论价格来自对走廊容量的有限差分探测还是来自关键链代理, 它收取的都是一次后果已被全额支付的占用费——那笔后果就在让行车辆自己的到达时刻里。任何以完整无冲突解码作为适应度的架构都具有这一性质, 而该性质的经验形态就是“价格是惰性的”。于是决定比较结果的, 变成了为一个惰性信号配备行动机器所需的代价。由此得到的设计推论比我们原先想要的更锐利: 在丰富一个层间接口之前, 先检查接收层的目标函数是否已经内部化了该接口要携带的东西; 若已内部化, 这种丰富不只是无益, 而是严格有害的。

5.6.3 机制层面的解释: 代价与增益的稀释. 上面的比较确立了凭证与错峰两个机制未能抵偿其代价, 但没有解释为什么。两项测量给出了解释, 而这两项解释同时也说明了预约表试探为何需要先降本才能显现价值。

算力去了哪里。把搜索的解码次数分开计数可以看到, 冲突制导局部搜索消耗了全部解码次数中的很大一部分, 而在它所解码的邻居里, 能真正改进完工时间的比例低于五分之一。于是相当大的一份预算被花在一个“命中率不足百分之二”的邻域上。要紧的比较不是那些改进是否真实——它们是真实的——而是同样这些解码次数若还给种群循环会不会产出更多。在这样的命中率下确实会。

这个低命中率不是实现上的缺陷, 我们为确认这一点花了一些功夫。该邻域按构造既小又有指向, 每轮至多 $L+2=7$ 个候选, 且每一个都由下层指名; 一个有指向的邻域仍然命中率低, 意味着凭证所指认的那些工序, 改派或错峰之后并不能缩短关键链。我们尝试过三种针对性修补——按路径上的预期让行而非按理想距离为改派候选打分、把错峰算子的相邻交换换成定向插入、以及接纳“完工时间不变但总让行下降”的平台移动——每一种都在同评价次数下与未修改的算子独立比较, 无一达到显著。我们报告这次尝试, 因为它界定了结论的范围: 失效在信号本身, 而不在我们如何使用信号。

为什么信号弱: 沿关键链的稀释。一份冲突凭证指明了某次让行事件的走廊、时段与对手任务, 这恰是消除那一次事件所需的信息。但完工时间是经由一条链达成的, 消除一处走廊延误通常会把另一个约束项成紧约束——某道上游工序、某台机械臂, 或另一条走廊——因此实现的改进只是被消除延误的一个分数。我们自己的接受准则之所以在每次接受移动后都重新提取关键链, 正是出于这个原因。所以, 当瓶颈只是众多量级相当的瓶颈之一时, 一个能指名瓶颈的反馈, 其价值远低于它的量级所暗示的。

这如何解释预约表试探的两次不同结论。稀释同样作用于预约表感知的派遣: 查表挑出的是当前这个任务送达最早的车, 这是一个局部改进, 其对完工时间的影响也要经过同一条链被稀释。这正是它在降本之前难以显现的原因——一个被稀释后仍为正、但幅度不大的收益, 遇上 12.2 倍的开销, 净效果自然落进噪声里 (第 5.5 小节引用的那批数据即是如此)。第 4.5 小节把开销压到 5.23 倍之后, 同一个被稀释的收益就浮出了噪声。

由此得到本文在机制层面的核心判断, 它比任何具体算子都更可迁移: 在固定预算下, 一个机制能否成立, 取决于“它被稀释后剩下的收益”与“它每次评价所增加的开销”之比, 而这两者都是可以分别测量、并可分别改进的。凭证机制失效在分子——稀释后所剩太少, 且三次针对性修补都无法把它抬起来; 派遣机制起初失效在分母, 而分母是在不改变输出的前提下压低的。这个区分是我们选定后者、放弃前者的全部理由。

5.7 代价律与降本效果

前面各小节反复用到“成本与评价次数是同一份资源的两种花法”这一论点。本小节把它本身作为对象来测, 因为本文承重机制的成立与否完全悬于此: 同一个机制, 在降本之前净效果落在噪声里, 在降本之后显著为正。

两种预算口径都不是中性的。图 5 给出各配置的单次评价成本与同预算内实际完成的评价次数。成本跨越两个数量级, 因此在同挂钟下便宜的配置能搜得多得多。这是为什么按相同代数比较是不可辩护的, 也是为什么对齐时间同样不是一个免费的选择。

图 6 把同一组比较在两种口径下各做一遍。同代数口径不为“机制多做的那部分工作”收费, 因而系统性地偏袒复杂配置: 在它之下预约表试探对规则派车有明显增益, 而同挂钟口径下同一比较在降本之前几乎归零。两种口径都不错。同代数回答的是“该机制是否改进了一步搜索”, 同挂钟回答的是“该机制是否改进了一次搜索”; 前者是机制设计问题, 后者是是否值得部署的问题。不可辩护的是报告其中一个却把它描述成另一个。

代价律。派遣试探把每个运输任务的路由调用数从 2 抬到至多 $2N_A$, 故其开销随车队规模线性增长。剪枝所依赖的下界在争用越弱时越紧, 因而剪掉的候选越多; 反过来, 车队越大、争用越强, 剪枝的效力越低。第 4.5 小

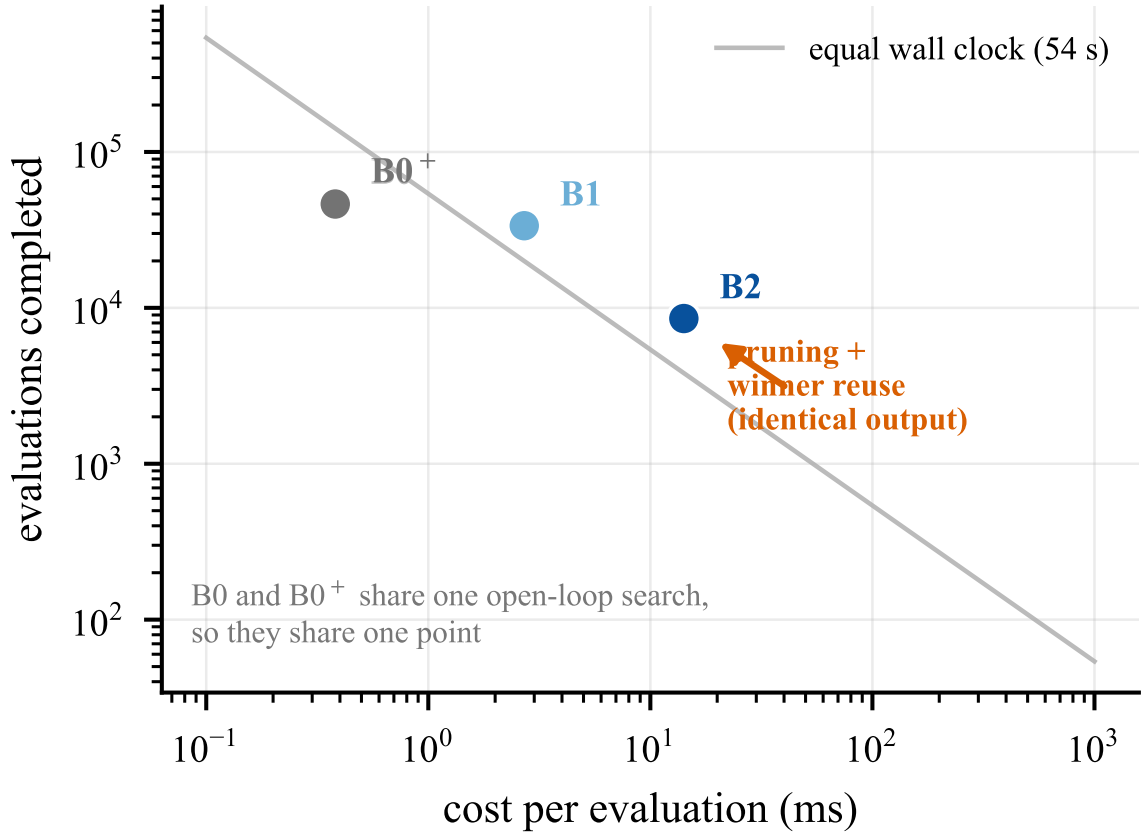
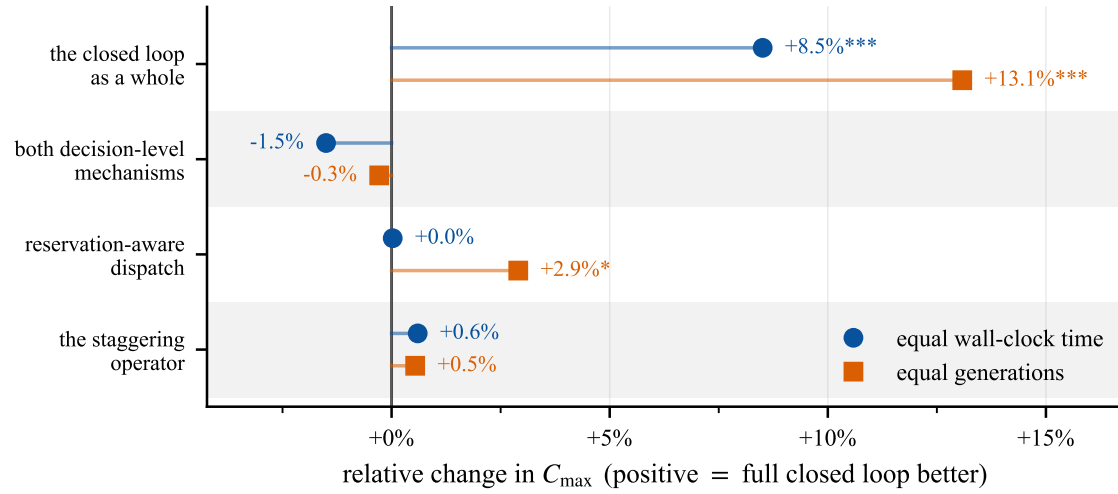


Fig. 5. 固定挂钟预算把“单次评价成本”与“完成的评价次数”锁成一条双曲线（灰线，由实测预算画出），各配置只能沿它移动。这就是本文反复引用的兑换率：一个机制必须在每次评价里买到的，多于它在评价次数上让出的。B0 与 B0⁺ 共用同一次开环搜索，故共用一个点，差别只在执行方式。橙色箭头是降本消融的两档——它是唯一一种不改变输出就能沿该线移动的干预（命题 3），因此这一段位移是纯粹的算力节省。

节的等价性自检已经显示出这个方向：六格中调用削减率最高的是最不拥堵的一格（61.2%），最低的是车队最大的一格（42.2%）。两个趋势合起来给出机制的适用区间——在车队足够大且争用足够强时，试探的开销增长会追上它的收益，这正是第 5.5 小节 B 族要定位的上界。

降本值多少：一个只动降本的受控对照。把“降本”与“机制”分离开来度量是必要的，因为两者在时间上是先后引入的，若只并列两个批次的结果，无法排除算例集与基线口径的差异。我们因此做了一个只动降本这一项的对照：同算例、同种子、同预算，两档分别开启与关闭剪枝与胜者路径复用。

这个对照之所以干净，是因为命题 3 保证两项优化不改变输出。我们先把这条命题兑现为可观测的事实：在六个算例上各取六条随机染色体，两档给出的完工时间与派车序列逐位相同，共 36 组无一例外；而路由调用数合计减少 47.4%。既然输出相同，关掉优化唯一的作用就是变慢，那么同挂钟预算下的完工时间差值就只来自省下的算力，不掺任何机制变化。



4 instances, 10 seeds, paired Wilcoxon; * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$. Batch measured before the cost reduction of Sec. 4.5.

Fig. 6. 同一组比较在两种预算口径下的读数。每一行按它所隔离的机制命名,而非按被比较的配置命名。两个标记之间的落差,就是表观增益中属于记账假象的那一部分。“预约表感知派遣”一行在同代数下为 +2.9% 且显著,在同挂钟下归零。该批次运行于第 4.5 小节的降本之前,因此这一行同时是下文那个受控对照的动机。

表 8. 只改变剪枝与胜者路径复用的受控对照。两档输出逐位相同 (已过等价性自检), 故差值只来自省下的算力。同算例、同种子、同挂钟预算, $n = 60$ 配对。

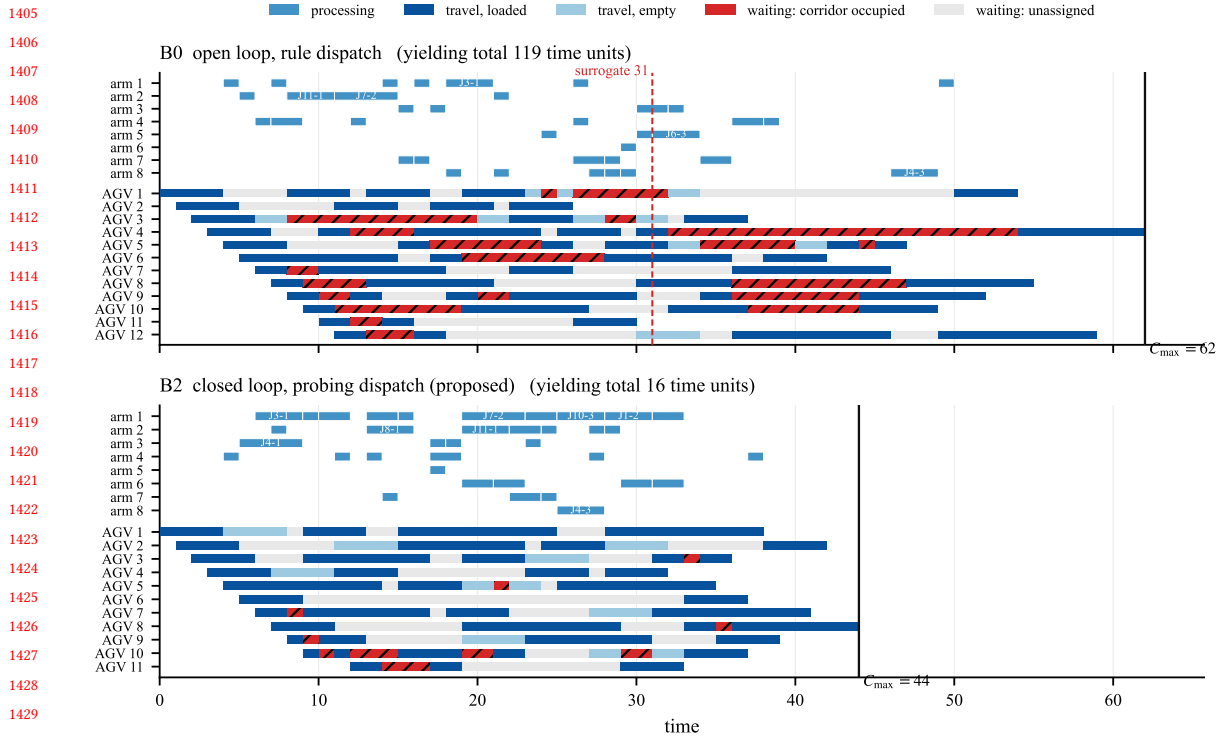
算例	争用强度	关	开	$\Delta C_{\max}$	评价次数比	加速比
A funnel	31.9%	44.80	42.10	-6.03%	2.21×	2.23×
A high	30.9%	41.60	37.70	-9.37%	2.40×	2.41×
A low	11.8%	60.50	57.00	-5.79%	2.79×	2.81×
B NA/NM 0.5	15.8%	66.70	65.10	-2.40%	2.11×	2.11×
B NA/NM 1.0	30.2%	51.90	48.90	-5.78%	2.31×	2.31×
B NA/NM 2.0	32.4%	44.50	42.20	-5.17%	1.98×	2.01×
配对合计 (60 配对)	—	—	—	-5.60%***	50/0/10, $< 10^{-4}$	

表 8 给出结果: 关 $\rightarrow$ 开使完工时间再降 5.60% (50 胜 / 0 负, $p = 0.0000$), 同预算内评价次数为 2.23 倍, 单次评价加速 2.33 倍。它把两个批次之间那处表面矛盾接上了最后一环: 同一个机制, 在降本前的矩阵批次上净效果落在噪声里 (-0.74%, 0/16 格显著), 在降本后的阶梯批次上显著为正, 而中间这一步没有改变它做出的任何一个选择。

5.8 案例分析: 甘特图与关键链归因

汇总统计量说明机制平均起来有效, 但不说明它在一次具体求解里做了什么。本小节在单个漏斗算例上把 B0 与 B2 的解并排画出, 并沿关键链做归因, 使前面的论证落到可以逐条核对的事件上。

图 7 是两个解的甘特图, 机械臂与车辆画在同一时间轴上, 车辆行的阴影段区分空载与满载, 让行等待单独着色。可以直接读出的三点是: 开环解中让行等待成段地出现在站台附近的走廊上, 且集中在少数几辆车上; 闭环解把这些等待大幅削去, 代价是若干道工序被指派到了并非最快的机械臂; 以及闭环解的车辆利用更为均衡。第



instance A funnel, seed 42, contention 31.9%

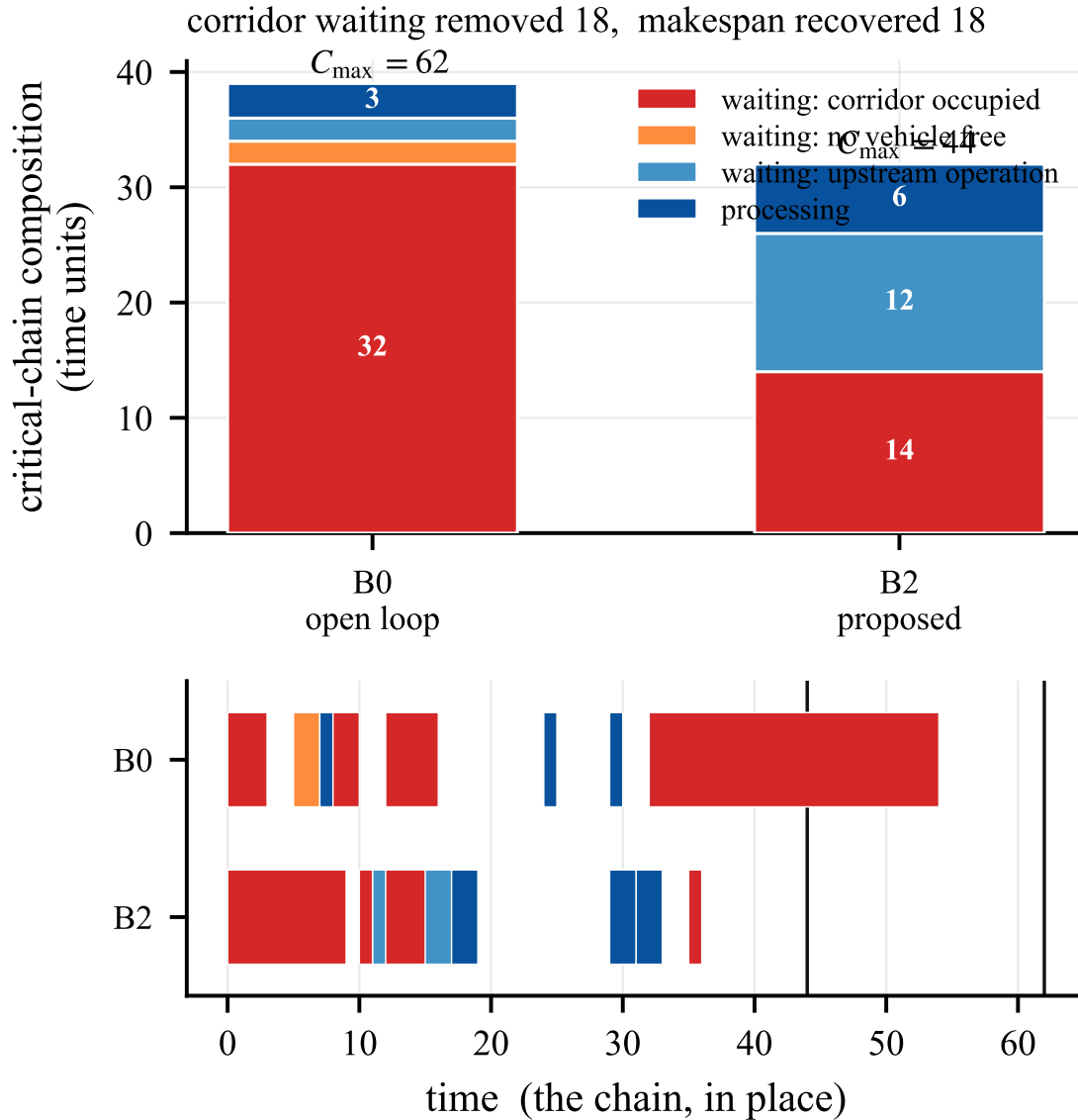
Fig. 7. 同一漏斗算例上 B0(上) 与 B2(下) 的解。机械臂与车辆共用时间轴; 车辆行区分空载段、满载段, 以及两类等待——按成分分开着色是本图唯一要紧的设计: 同一运输任务两段之间的空隙是车被前方占用的走廊拦在停靠位上, 即让行, 也就是本文所处理的那种延误; 不同任务之间的空隙则是车无活可干, 属于车队规模的症状。两者若同色, 读者就会把其中一种归因成另一种。图内文字为英文以便与代码输出一致。

三点是前两点的结果而非另一个机制——它说明”用加工时间买邻近性”这一权衡(14) 在实际解中确实被行使了。

图 8 沿关键链做归因, 把每一环按表 3 的五类标注着色。它使第 5.6.3 小节的稀释论证可视: 在 B0 的链上走廊环占据显著比例, 而在 B2 的链上它们大体被消除, 顶上来的是机械臂环与上游工序环。这正是”消除一处走廊延误会把另一个约束顶成紧约束”的直接图示, 也解释了为何指名瓶颈的反馈其价值低于它的量级。

6 结论

本文研究异构机械臂加工与 AGV 无冲突路由的一体化调度, 其前提是文献通常回避的那一条: 行驶时间 由路由决策产生, 而不是从一张矩阵里读出来。在这一设定下, 我们没有停留在”耦合两层是否有益”这个已有答案的问题上, 而是问了一个更具体的: 当预算固定、任何信息都要用放弃掉的评价次数来支付时, 哪一种层间信息值得它的代价, 以及如何让它付得起。



instance A funnel, seed 42

Fig. 8. 关键链归因。上: 两档关键链的构成, 每一环按表 3 的标注着色; 标题直接并列“被消除的走廊等待”与“实际收回的完工时间”, 二者之差即被顶上来的约束吸收掉的部分。下: 同一条链在时间轴上的原位, 以表明上方的柱是一条链的分解, 而非若干互不相关事件的直方图。闭环解中走廊环大体消失, 顶上来的是机械臂与上游工序——这就是稀释效应的直接图示。

6.1 结论与贡献

(1) 一个双反馈通路的闭环双层框架。上层以 memetic 算法搜索指派与排序, 下层以时间窗预约做无冲突路由; 每一代每一个个体的适应度都是下层完成全部运输任务规划之后的完工时间, 因此上层从不优化代理目标。命

题 2 与引理 1 保证任何染色体都可解码、评价相对第 3 节的冲突模型是精确的, 且不需要任何修复算子。把拥堵搬进搜索目标这一步, 在规则派车下值 18.86% ($p = 0.0000$), 在试探派车下值 17.75% ($p = 0.0000$)。

(2) 预约表感知的车辆派遣, 以及使它付得起的降本。选车是整个框架中最后一处可以用估计代替事实的环节, 我们把它闭合: 选车前对预约表做两段式试探, 用实际可达的送达时刻代替理想最短路的估计。在闭环内部它值 2.45% ($p = 0.0054$)。这个机制的关键困难不在于设计而在于代价——它使单次评价贵 12.2 倍, 足以在同挂钟预算下把自身收益吃光。我们给出两项优化 (可采纳下界剪枝与胜者路径复用), 并证明它们不改变输出 (命题 3), 又以逐位等价性自检加以实证; 它们把路由调用减少 47.4%, 把成本倍数压到 5.23。因此本文对这一机制的主张是完整的: 它有效, 而且我们说明了它为何一度看不出有效, 以及如何让它有效。

(3) 主效应与有上界的次效应, 以及支撑该结论的实验协议。闭环是主效应; 预约表感知派遣在开环与闭环下都显著 (4.59%, $p = 0.0000$; 2.45%, $p = 0.0054$), 但大车队两格上后者会被开销反噬。端到端 21.35% 接近独立乘积 22.6% (近加性, 无正交互); 开环试探单独远不能代替闭环。这对“先排产、后解冲突”给出结构性判断: 第二阶段能显著挽回一部分, 但主效应仍留在桌面上。支撑它的是 2×2 析因与统一执行器口径: 无论计划从哪来, 都必须经同一个无冲突路由器执行、过同一个校验器, 报的都是可实现的完工时间。

6.2 主要洞见

有三条判断的适用范围比本文的具体算子更宽, 我们认为它们是本文更值得带走的部分。

接口丰富之前, 先查接收层是否已经内部化了它要携带的东西。我们完整实现了走廊时段定价, 它在同挂钟预算下系统性地有害。原因不是估计器不好, 而是架构性的: 由争用造成的延误已经完整体现在让行车辆自己的到达时刻里, 并经 (10) 传播进完工时间被适应度计入; 再对同一次占用收一次价格等于把它数了两遍。任何以完整无冲突解码作为适应度的框架都具有这一性质, 其经验形态就是“价格是惰性的”。而一旦引入价格作为第二判据, 引理 1 的全序占优即被摧毁, 单标签搜索必须让位给多标签搜索——这笔钱无论是否真的收取价格都要付。

区分“改进一个决策”与“改进一次测量”。指名瓶颈的反馈买到的是局部改进, 而完工时间是经由一条链达成的, 消除一处延误通常把另一个约束项成紧约束, 因此实现的改进只是被消除延误的一个分数。修正目标函数则不同: 它不作用于链上任何一环, 而是改善搜索将要比较的每一个候选解的排序, 因此无须经受稀释。这个不对称解释了为什么冲突凭证制导的局部搜索在三次针对性修补之后仍不划算, 而它也同时解释了预约表试探为何需要先降本——被稀释后仍为正但幅度不大的收益, 遇上足够大的开销就会落进噪声。

在固定预算下, 机制的成立与否是一个可分解、可分别改进的比值。分子是“稀释后所剩的收益”, 分母是“每次评价所增加的开销”。凭证机制失效在分子, 而分子难以人为抬高; 派遣机制起初失效在分母, 而分母可以在不改变任何输出的前提下压低。这个区分不仅是本文的取舍依据, 也给出了一条可操作的诊断顺序: 先测一个机制的开销倍数与其被稀释后的收益, 再决定是改进它还是放弃它。它同时意味着一件容易被忽略的事——工程上的降本可以是方法学上的贡献, 因为在同预算口径下降本与增效是同一件事。

6.3 局限

(1) 算例族。全部算例格共用一个规模与一套布局模板。我们变化的是分析判定为关键的那些因子 (布局、车臂比、柔性), 但规模与拓扑类型是固定的, 我们不就机制如何随规模变化作任何主张。

(2) 没有公开基准的对比。标准 FJSP_PT 基准假定行驶时间为常数矩阵、不提供通道网络, 因而无法表达车辆争用; 在其上运行需要先删掉本文所讨论的那条约束。一个退化的比较——把本文上层在关闭路由的条件下与已发表结果相比——检验的是遗传算法而非本框架, 我们没有做。

(3) 下界质量与最优距离。复合下界是松的, 与最优已知解之间的间隙中位数为 53%。它的作用是数量级核查与回归防护, 不是最优性证书; 这么大的间隙无法区分“搜索失败”与“下界松弛”, 因此我们不就解与最优解的接近程度作任何主张。

(4) **单一实现**。所有配置共用一套代码,这消除了配置之间的实现方差,但也意味着绝对完工时间反映的是该实现的质量。各配置之间的差值是内部比较,不受此影响;与已发表方法的对比会受影响。

(5) **负面结果的作用域**。三个机制都是在一个由纯 Python 实现标定的预算下被判定的。我们的比较所测量的是“成本与评价次数之间的兑换率”,且它是在共享同一实现的配置之间测得的,故对一个常数倍的加速是稳健的;但它对各部件相对成本的改变并不稳健。这正是本文降本一节的意义所在——我们已经在派遣上演示了改变相对成本可以翻转结论。因此我们的结论应被理解为:这些机制在当前代价下不划算,而非它们不可能划算。

(6) **收敛性**。交换乘子或价格的双层方案有时可被证明收敛到不动点,本框架不能,我们也不作此主张。下层是优先级式而非联合最优,上层是非凸组合空间上的随机搜索,层间传递的是证据而非可充当 Lyapunov 函数的量。

6.4 未来工作

增量式评价。解码器的状态严格向前流动,因此只在序列某位置之后被扰动的邻居,其之前的全部事件与预约都不变。一个能够只重解被扰动部分的增量式评价器,会恰好降低本文中失效的那些机制的代价,冲突制导领域是在它之下最值得重测的第一个候选。按上一小节的比值框架,这是从分母一侧再做一次尝试。

链可替代性作为算例维度。稀释论证预言:关键链越不可替代——即车间只有一个主导瓶颈而非许多量级相当的瓶颈——制导型反馈就越有价值。本文的布局族变化的是争用的可避免性而非链的可替代性,且我们为量化前者引入的漏斗份额指标未能区分它本要区分的两个族。一个按链可替代性参数化的算例族,配上一个经过“它声称度量的那个操纵”检验过的指标,才能真正检验这条解释,而不只是与之不矛盾。

小规模精确解。已有工作表明基于逻辑的 Benders 分解可以为该问题类给出精确解 [8]。小规模精确解可以确定间隙中有多少是可回收的,也可以检验那些在启发式搜索下不划算的机制,在搜索更强时是否依然不划算。

动态运行。本文的建模假定加工时间已知、工件集固定。在线到达与车辆故障会改变本文所报的这套算术,因为那时一个机制的价值还包括它修复排程的速度,而不只是它构建排程的质量;一个必须从头重解的廉价评价器,与一个能把损害局部化的制导机制相比,结论可能不同。

References

- [1] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production-Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.
- [2] M. R. Garey, D. S. Johnson, and Ravi Sethi. 1976. The Complexity of Flowshop and Jobshop Scheduling. *Mathematics of Operations Research* 1, 2 (1976), 117–129. doi:10.1287/moor.1.2.117
- [3] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139
- [4] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543
- [5] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355
- [6] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352
- [7] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526
- [8] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342
- [9] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795
- [10] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*.

1613 Received 27 November 2025; revised 26 February 2026; accepted XX XX 2026

1614

1615

1616

1617

1618

1619

1620

1621

1622

1623

1624

1625

1626

1627

1628

1629

1630

1631

1632

1633

1634

1635

1636

1637

1638

1639

1640

1641

1642

1643

1644

1645

1646

1647

1648

1649

1650

1651

1652

1653

1654

1655

1656

1657

1658

1659

1660

1661

1662

1663

1664

Manuscript submitted to ACM